

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

GA – EXPOSICIÓN SEGUNDO CORTE: HERRAMIENTAS CASE (StarUML)

MATERIA:

INGENIERÍA DE REQUERIMIENTOS

EQUIPO B

INTEGRANTES:

CORDOVA CARREÑO MAYRA LUCILA (**Investigadora**)
CI: 0750286775 - mcordovac2@uteq.edu.ec

GUTIERREZ ORTEGA GENESIS ADRIANA (**Titular / Líder / Modeladora UML**)
CI: 1250274477 - ggutierrez@uteq.edu.ec

NARANJO FLORES ANDERSON JEAMPIERE (**Verificador**)
CI: 1207412659 - anaranjof2@uteq.edu.ec

SISTEMA PROYECTO FIN DE CURSO:

MUNDIPETS (GUTIERREZ GENESIS - TITULAR PFC)

CURSO:

CUARTO SEMESTRE «B»

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERÓN, PhD.

REPOSITORIO GITHUB:

<https://github.com/GenessiGutierrez/PFC-MundiPets-MDD-EquipoB.git>

QUEVEDO – LOS RÍOS – ECUADOR

2026 – 2027 PPA

Índice

1.	Resumen	3
2.	Marco Teórico	3
2.1.	Model-Driven Engineering (MDE)	3
2.2.	Model-Driven Architecture (MDA) — CIM/PIM/PSM	3
2.3.	Model-Driven Development (MDD)	3
2.4.	Transformaciones M2M	4
2.5.	Transformaciones M2T	4
2.6.	Metamodelo	4
2.7.	Perfiles UML	4
2.8.	Plantilla / Template	4
2.9.	Forward, Reverse y Round-Trip Engineering	5
2.10.	Diagramas UML Admisibles como Origen	5
2.11.	Estado del Arte	5
3.	Justificación de la selección de StarUML	6
3.1.	Herramienta seleccionada y versión	6
3.2.	Justificación frente a los requisitos del PFC	6
3.3.	Comparación con alternativa descartada	6
4.	Descripción del subsistema del PFC modelado	7
4.1.	Contexto de MundiPets	7
4.2.	Módulo seleccionado	7
4.3.	Actores	7
4.4.	Requisitos funcionales	7
5.	Modelo UML de origen	9
5.1.	Diagrama de clases	9
5.2.	Relaciones UML	9
5.3.	Estereotipos	10
5.4.	Diagramas complementarios	10
5.5.	Validación del modelo en StarUML	10
6.	Configuración del Generador	11
6.1.	Plantilla y Parámetros	11
6.2.	Política de Regeneración y Decisiones de Mapeo	11
7.	Proceso de generación	12
7.1.	Ejecución y capturas	12
7.2.	Árbol del proyecto y trazabilidad clase-archivo	13
8.	Verificación del código generado	15
8.1.	Compilación y ejecución	15
8.2.	Correspondencia modelo-código	18
8.3.	Limitaciones detectadas	18
9.	Cierre del ciclo: round-trip engineering	20
9.1.	Cambio aplicado al modelo	20
9.2.	Evidencia antes y después	22
10.	Reparto del trabajo	23
11.	Conclusiones	24
12.	Referencias	25
13.	Anexos	26

Índice de figuras

1.	Diagrama de Clases del Módulo Gestión de Solicitud de Adopción “MundiPets”	9
2.	Diagrama Caso de Uso del Módulo Gestión de Solicitud de Adopción “MundiPets”	10
3.	Resultado de la validación del modelo en StarUML — sin errores	11
4.	Invocación del generador de código C# desde StarUML.	12
5.	Selección del modelo base para la generación de código.	13
6.	Archivos C# generados a partir del modelo UML de MundiPets.	13
7.	Árbol de archivos generado a partir de las clases UML del subsistema.	14
8.	Trazabilidad entre las clases del modelo UML y los archivos C# generados.	15
9.	Primera compilación del código generado, con errores de incompatibilidad de tipos.	16
10.	Segunda compilación después de adaptar los tipos UML al lenguaje C#.	16
11.	Compilación exitosa del código generado después de aplicar correcciones de compatibilidad.	17
12.	Ejecución de una prueba estructural sobre la clase csSolicitudAdopcion.	17
13.	Correspondencia entre la clase UML csSolicitudAdopcion y el archivo C# generado.	18
14.	Comparación entre el código original generado y la corrección manual de tipos.	19
15.	Corrección manual de expresiones de retorno incompatibles con métodos void.	20
16.	Clase csSolicitudAdopcion antes de aplicar el cambio de round-trip engineering.	21
17.	Código de csSolicitudAdopcion antes de la regeneración.	21
18.	Clase csSolicitudAdopcion después de incorporar el motivo de cancelación.	22
19.	Código de csSolicitudAdopcion después de regenerar el modelo modificado.	22
20.	Comparación del código antes y después de la regeneración.	23
21.	Contribuciones por integrante en GitHub (Insights → Contributors).	24
22.	Código generado — csUsuario.cs	26
23.	Código generado — csUsuarioRol.cs	26
24.	Código generado — csRol.cs	27
25.	Código generado — csMascota.cs	27
26.	Código generado — csHistorialMedico.cs	28
27.	Código generado — csPublicacion.cs	28
28.	Código generado — csDocumentoVeterinario.cs	29
29.	Código generado — csSolicitudAdopcion.cs	29
30.	Fragmento de code-generator.js — función getType()	30

Índice de tablas

1.	Niveles de abstracción de MDA	3
2.	Tipos de transformación M2M	4
3.	Lenguajes de transformación M2T	4
4.	Jerarquía de capas del metamodelo	4
5.	Mecanismos de extensión de los perfiles UML	4
6.	Tipos de plantilla	5
7.	Lenguajes de plantillas por herramienta	5
8.	Conceptos de ingeniería directa, inversa y de ida y vuelta	5
9.	Diagramas UML admisibles como origen de generación	5
10.	Técnicas de IA aplicadas a MDE	5
11.	Desafíos identificados en las transformaciones de modelos	5
12.	StarUML vs. Visual Paradigm	7
13.	Actores del módulo de Gestión de Solicitud de Adopción	7
14.	RF-05 — Gestionar solicitudes de adopción	8
15.	Requisitos funcionales complementarios del módulo	8
16.	Relaciones UML del módulo	9
17.	Estereotipo aplicado y su efecto en el código generado	10
18.	Parámetros de configuración utilizados.	11
19.	Decisiones de mapeo estructural.	12
20.	Correspondencia entre clases UML y archivos C# generados.	14
21.	Resultados de la compilación y ejecución del código generado.	17
22.	Correspondencia entre la clase UML csSolicitudAdopcion y el código generado.	18
23.	Limitaciones identificadas en el código generado por StarUML.	19

24.	Comparación de los elementos antes y después de la regeneración.	23
25.	Reparto del trabajo por sección.	23

1 Resumen

MundiPets es un sistema orientado a centralizar la información de mascotas y a facilitar los procesos de adopción y cruce responsable, conectando de forma organizada, transparente y segura a propietarios, adoptantes, interesados en cruce, refugios y veterinarias. El proyecto contempla, además, la incorporación de funcionalidades de apoyo basadas en inteligencia artificial alertas, recomendaciones preliminares y moderación de contenido que refuercen la calidad y seguridad de la información, sin sustituir la evaluación profesional veterinaria ni las decisiones definitivas sobre salud o reproducción de las mascotas. Como caso de aplicación del ciclo *Model-Driven Development* (MDD), se seleccionó la herramienta CASE StarUML (versión 7.1.0) para la construcción del modelo UML de origen y la generación automática de código fuente, tomando como subsistema representativo el módulo de Gestión de Solicitud de Adopción, dado su carácter central en el flujo de negocio de la plataforma. A partir del diagrama de clases de este módulo, compuesto por ocho clases del dominio, se configuró el generador *C# Code Generation and Reverse Engineering* de StarUML para producir código fuente en C#, con el fin de evidenciar la trazabilidad entre el modelo UML y el código producido, así como el cierre del ciclo mediante una prueba de *roundtrip* controlada entre el modelo y el código generado.

2 Marco Teórico

2.1 Model-Driven Engineering (MDE)

Model-Driven Engineering (MDE) es un paradigma de la ingeniería de software que posiciona a los modelos como artefactos de primera clase dentro del proceso de desarrollo, es decir, como elementos centrales y ejecutables, y no como simples documentos de apoyo. En este enfoque, el modelo se convierte en la fuente autoritativa del sistema, y las implementaciones se derivan de él mediante transformaciones automatizadas. Los modelos se definen con conceptos cercanos al dominio del problema, facilitando su comprensión y mantenimiento [1].

MDE busca aumentar la productividad y la calidad del software mediante la creación sistemática de modelos. Según estudios recientes, MDE ha demostrado ser efectivo en múltiples dominios: el 56.8 % de los estudios se centran en desarrollo general, el 21.6 % en sistemas embebidos, y el 13.5 % en aplicaciones web. A pesar de sus beneficios, su adopción enfrenta barreras como la falta de conocimiento y habilidades, y las dificultades con las herramientas disponibles [1].

2.2 Model-Driven Architecture (MDA) — CIM/PIM/PSM

Model-Driven Architecture (MDA) es una iniciativa del Object Management Group (OMG) que estructura el desarrollo en tres niveles de abstracción:

Tabla 1: Niveles de abstracción de MDA

Nivel	Descripción
CIM (Computation Independent Model)	Modelo de requisitos y contexto del negocio, sin detalles de implementación. Usa BPMN [2].
PIM (Platform Independent Model)	Estructura y funcionalidad del sistema de manera abstracta. Usa UML [2].
PSM (Platform Specific Model)	Información técnica de implementación en una plataforma particular (Java/Spring) [2].

Los requisitos del CIM deben ser trazables hasta el PIM y PSM. Las transformaciones se realizan en dos etapas: CIM→PIM y PIM→PSM. La transformación CIM→PIM es menos frecuente debido a las diferencias fundamentales entre ambos niveles [2].

2.3 Model-Driven Development (MDD)

Model-Driven Development (MDD) es la aplicación práctica de MDE, donde los modelos guían la construcción del sistema y el código se genera automáticamente. MDD mejora la productividad y la calidad del software, especialmente en proyectos complejos. Permite que expertos del dominio participen en el desarrollo mediante abstracciones de alto nivel [3].

2.4 Transformaciones M2M

Las transformaciones Model-to-Model (M2M) producen un modelo destino a partir de un modelo origen [4]. Pueden ser:

Tabla 2: Tipos de transformación M2M

Tipo	Dirección
Horizontal	Mismo nivel de abstracción
Vertical	Diferente nivel de abstracción (ej. PIM→PSM)

El estándar OMG es QVT. Otros lenguajes incluyen ATL y EGL. La calidad de estas transformaciones es crítica para la fidelidad de los modelos generados. Los desafíos incluyen el manejo de colecciones, recursión y condiciones lógicas [4].

2.5 Transformaciones M2T

Las transformaciones Model-to-Text (M2T) producen código fuente a partir de un modelo. El estándar OMG es MOFM2T, implementado por Acceleo. Otros lenguajes incluyen Xtend y EJS (usado por StarUML) [5].

Tabla 3: Lenguajes de transformación M2T

Lenguaje	Estado	Características
Acceleo	Mantenimiento pasivo	Implementa MOFM2T
Xtend	Futuro incierto	Tipado estático, integración Java
EJS	Activo	Basado en JavaScript, usado por StarUML

2.6 Metamodelo

Un metamodelo define la sintaxis abstracta de un lenguaje de modelado. UML se define sobre MOF [5]. La jerarquía de cuatro capas:

Tabla 4: Jerarquía de capas del metamodelo

Capa	Descripción	Ejemplo
M3	Meta-metamodelo	MOF
M2	Metamodelo	Metamodelo UML
M1	Modelo	Diagrama de clases de MundiPets
M0	Instancias	Objetos en ejecución

Ecore es la herramienta líder (70.3 % de los estudios). La composición de modelos se clasifica en: basada en operador, basada en aspectos y basada en reglas [6].

2.7 Perfiles UML

Los Perfiles UML extienden el metamodelo UML sin modificarlo, mediante:

Tabla 5: Mecanismos de extensión de los perfiles UML

Mecanismo	Descripción
Estereotipo	Nueva variante de elemento UML
Valor etiquetado	Propiedades adicionales
Restricción	Reglas OCL
Extensión de metacalse	Extiende metaclasses específicas

Los perfiles más comunes se enfocan en sistemas embebidos, arquitectura empresarial, seguridad y desarrollo web [5].

2.8 Plantilla / Template

Las plantillas permiten la reutilización y parametrización de modelos [7]. Existen dos tipos:

Tabla 6: Tipos de plantilla

Tipo	Descripción
Plantillas de modelo	Modelos parametrizados que capturan patrones estructurales
Plantillas de transformación (M2T)	Fragmentos de texto parametrizados para generar código

Las plantillas M2T combinan: (1) texto fijo, (2) instrucciones de control, y (3) referencias al modelo [8].

Tabla 7: Lenguajes de plantillas por herramienta

Herramienta	Lenguaje de plantillas
StarUML	EJS
Eclipse/Acceleo	MOFM2T
Enterprise Architect	Plantillas propietarias
Visual Paradigm	Template Editor

2.9 Forward, Reverse y Round-Trip Engineering

Tabla 8: Conceptos de ingeniería directa, inversa y de ida y vuelta

Concepto	Descripción
Forward Engineering	Generación de código a partir del modelo
Reverse Engineering	Creación de modelo a partir del código existente
Round-Trip Engineering (RTE)	Sincronización bidireccional entre modelo y código

El RTE requiere: (1) preservación de información, (2) sincronización bidireccional, y (3) mecanismos de actualización (instantánea o bajo demanda). Existen bloques protegidos para preservar código escrito a mano durante la regeneración [9, 10].

2.10 Diagramas UML Admisibles como Origen

Tabla 9: Diagramas UML admisibles como origen de generación

Diagrama	Obligación	Uso
Clases	Obligatorio	Base para generación estructural [9, 11].
Estados	Recomendado	Máquinas de estados (patrón State) [9, 11].
Secuencia	Recomendado	Interacciones entre objetos [9, 11].
Actividades	Recomendado	Flujos de trabajo y lógica de negocio [9, 11].

2.11 Estado del Arte

Adopción industrial

Los principales desafíos en MDE son: escalabilidad, interoperabilidad, integración con metodologías ágiles y evolución de lenguajes. El 80 % de los enfoques se aplican a ejemplos académicos, no industriales [1].

Integración con IA y Machine Learning

Tabla 10: Técnicas de IA aplicadas a MDE

Técnica	Aplicación
NLP	Automatización del análisis de requisitos [2].
ML	Generación de modelos y clasificación [2].
LLM	Asistencia en desarrollo de transformaciones [2].

ModelSet contiene 5,466 metamodelos Ecore y 5,120 modelos UML para entrenar modelos de ML [2].

Desafíos en Transformaciones

Tabla 11: Desafíos identificados en las transformaciones de modelos

Desafío	Descripción
Complejidad	Manejo de colecciones, recursión y condiciones [12].
Detección de errores	Falta de técnicas automáticas [12].
Oráculos de prueba	Dificultad para determinar resultados esperados [12].
Debugging	Limitado soporte para depuración paso a paso [12].

Herramientas

StarUML: CLI para generación automatizada con EJS [13].

Enterprise Architect: Plantillas personalizables [13].

Visual Paradigm: Round-trip engineering [13].

Eclipse Papyrus + Acceleo: Estándares OMG [13].

3 Justificación de la selección de StarUML

3.1 Herramienta seleccionada y versión

Para el desarrollo del ciclo Model-Driven Development (MDD) del PFC MundiPets, el equipo seleccionó StarUML (MKLabs, versión 7.1.0), herramienta CASE que implementa el paradigma de Model-Driven Architecture (MDA) para transformar modelos en código de forma sistemática [14, 15]. StarUML figura explícitamente entre las herramientas admisibles según la guía de la actividad, que exige la entrega del proyecto nativo `.mdj` junto con los archivos de configuración de la extensión de generación utilizada.

3.2 Justificación frente a los requisitos del PFC

La selección de StarUML se sustenta en la correspondencia entre sus capacidades técnicas y los requisitos tecnológicos definidos para MundiPets:

- **Lenguaje objetivo y framework.** Para el módulo de Gestión de Solicitud de Adopción, StarUML dispone de la extensión *C# Code Generation and Reverse Engineering*, que transforma directamente las clases, atributos y operaciones del diagrama UML en clases C#, sin requerir configuración adicional de plantillas. La literatura advierte que la fidelidad de este tipo de generadores frente a la semántica completa del modelo suele ser limitada [17], por lo que se complementa con revisión manual (Secciones 6 y 8).
- **Escala del subsistema.** El módulo de Gestión de Solicitud de Adopción se modeló con 8 clases, superando el mínimo de 6 exigido. StarUML ofrece un editor de clases con atributos tipados, operaciones, visibilidad, cardinalidad y estereotipos –criterios estándar de comparación de herramientas CASE UML [18]–, suficiente para esta escala sin la complejidad de herramientas empresariales.
- **Estereotipos y perfiles.** Se aplicó el estereotipo «Entity» del *UML Standard Profile* de StarUML a las clases del dominio, como marca de clasificación semántica dentro del modelo. Sin embargo, la revisión del código generado (Sección 8) confirmó que la extensión *C# Code Generation and Reverse Engineering* no traduce este estereotipo a ninguna construcción del lenguaje C# no genera atributos, comentarios ni marcas equivalentes, por lo que su función es exclusivamente documental sobre el modelo UML y no forma parte del mecanismo de generación de código evaluado en este trabajo.
- **Licencia y curva de aprendizaje.** StarUML tiene licencia de evaluación gratuita e interfaz de alcance moderado. La simplicidad de configuración y el costo de licenciamiento son criterios determinantes en la selección de herramientas académicas [18].

3.3 Comparación con alternativa descartada

Como alternativa se evaluó y descartó Visual Paradigm, herramienta igualmente admisible según la guía de la actividad, cuyo soporte para la generación de mapeos ORM es relativamente más maduro que el de otras herramientas comparadas [17], aunque con limitaciones de fidelidad semántica similares al resto de generadores analizados.

Tabla 12: StarUML vs. Visual Paradigm

Criterio	StarUML (seleccionada)	Visual Paradigm (descartada)
Lenguajes objetivo típicos	Java, C#, C++, TypeScript, Python	Java, C#, C++, Python, VB, ActionScript
Generación de código	Generadores oficiales por extensión; la extensión de C# utilizada emplea una plantilla fija embebida en el complemento, no configurable desde la interfaz (Sección 6.1)	Instant Generator robusto con plantillas ORM/JPA/Hibernate preconfiguradas
Evidencia exigida por la guía	Proyecto .mdj + archivos de configuración de la extensión	Proyecto .vpp + evidencias de la corrida
Adecuación al alcance del módulo (8 clases de dominio)	Suficiente; interfaz simplificada sin sobrecarga de configuración	Mayor capacidad, pero orientada a proyectos de mayor escala empresarial, con configuración más compleja de lo requerido
Curva de aprendizaje	Menor; el equipo reportó facilidad de uso durante la práctica	Mayor complejidad funcional
Licencia	Evaluación gratuita disponible	Licencia comercial; versión de comunidad con funcionalidades limitadas

Si bien Visual Paradigm ofrece plantillas ORM/JPA/Hibernate preconfiguradas [17], el equipo priorizó StarUML por su menor complejidad de configuración para el alcance del módulo (8 clases) y por la simplicidad de invocar el generador directamente desde la interfaz sin necesidad de definir mapeos adicionales [16], evitando funcionalidades de Visual Paradigm orientadas a proyectos de mayor escala no aprovechables en este PFC.

4 Descripción del subsistema del PFC modelado

4.1 Contexto de MundiPets

MundiPets es una plataforma orientada a centralizar la información de mascotas y a facilitar los procesos de adopción y cruce responsable, conectando de forma organizada, transparente y segura a propietarios, adoptantes, interesados en cruce, refugios y veterinarias. El sistema incorpora componentes de apoyo basados en inteligencia artificial evaluación preliminar de compatibilidad, validación de imágenes y moderación de contenido sin reemplazar la evaluación profesional veterinaria ni las decisiones definitivas sobre la salud o reproducción de las mascotas. El acceso a la plataforma se gestiona mediante roles diferenciados, y la información sensible de propietarios y mascotas se visualiza únicamente conforme a los permisos definidos para cada tipo de usuario.

4.2 Módulo seleccionado

El módulo seleccionado es Gestión de Solicitud de Adopción (RF-05 de la ERS de MundiPets), central en el flujo de negocio al articular la interacción entre adoptante y propietario/refugio desde la postulación hasta la resolución de la solicitud. La entidad *SolicitudAdopcion* tiene un ciclo de vida definido: se genera en estado *Pendiente* al enviarse sobre una mascota disponible; pasa opcionalmente a *En revisión* mientras se evalúan las condiciones del solicitante; y se resuelve como *Aceptada*, *Rechazada* o *Cancelada*. Esta trazabilidad de estados sustenta el diagrama de clases (Sección 5) y el diagrama de casos de uso del módulo (Sección 5.4).

4.3 Actores

La Tabla 13 presenta los actores que interactúan directamente con el módulo de Gestión de Solicitud de Adopción, describiendo su rol específico dentro del flujo del proceso.

Tabla 13: Actores del módulo de Gestión de Solicitud de Adopción

Actor	Rol en el módulo
Adoptante	Consulta la publicación de una mascota disponible, envía la solicitud de adopción ingresando su justificación y las condiciones de su hogar, y recibe la notificación de la decisión tomada.
Propietario o refugio	Recibe la solicitud, revisa la información autorizada del adoptante y decide sobre su aceptación, rechazo o cancelación.

4.4 Requisitos funcionales

La Tabla 14 presenta la especificación completa del requisito funcional central del módulo (RF-05), siguiendo el formato de atributos utilizado en el ERS de MundiPets.

Tabla 14: RF-05 — Gestionar solicitudes de adopción

Atributo	Valor
Identificador	RF-05
Descripción	El sistema deberá permitir que un usuario interesado envíe solicitudes de adopción y que el propietario pueda aceptarlas o rechazarlas después de revisar la información del solicitante.
Actores	Adoptante, Propietario de mascota (o refugio)
Entradas	Solicitud de adopción, información del solicitante (justificación, condiciones del hogar)
Salidas	Solicitud registrada y estado de aceptación o rechazo
Precondiciones	La mascota debe estar disponible para adopción; el adoptante debe estar autenticado y no poseer una solicitud activa sobre la misma publicación
Postcondiciones	La solicitud queda registrada con su estado correspondiente
Prioridad (MoSCoW)	Must

Nota: el identificador RF-05 corresponde a la numeración original definida en el documento ERS/SRS parcial de MundiPets, por lo que se mantiene sin modificación para preservar la trazabilidad con la documentación de requisitos del proyecto.

La Tabla 15 resume los requisitos funcionales complementarios que interactúan con el módulo de Gestión de Solicitud de Adopción sin constituir su núcleo funcional.

Tabla 15: Requisitos funcionales complementarios del módulo

ID	Nombre	Prioridad	Vínculo con el módulo
RF-07	Sistema de mensajería entre usuarios	Could	Permite la comunicación entre el adoptante y el propietario o refugio mientras la solicitud de adopción permanece activa.
RF-15	Consultar el historial de procesos de adopción y cruza	Could	Permite al propietario consultar el historial de solicitudes asociadas a sus mascotas, incluyendo estado y fecha de resolución.

Nota: los identificadores RF-07 y RF-15 corresponden igualmente a la numeración original del ERS/SRS parcial de MundiPets.

5 Modelo UML de origen

5.1 Diagrama de clases

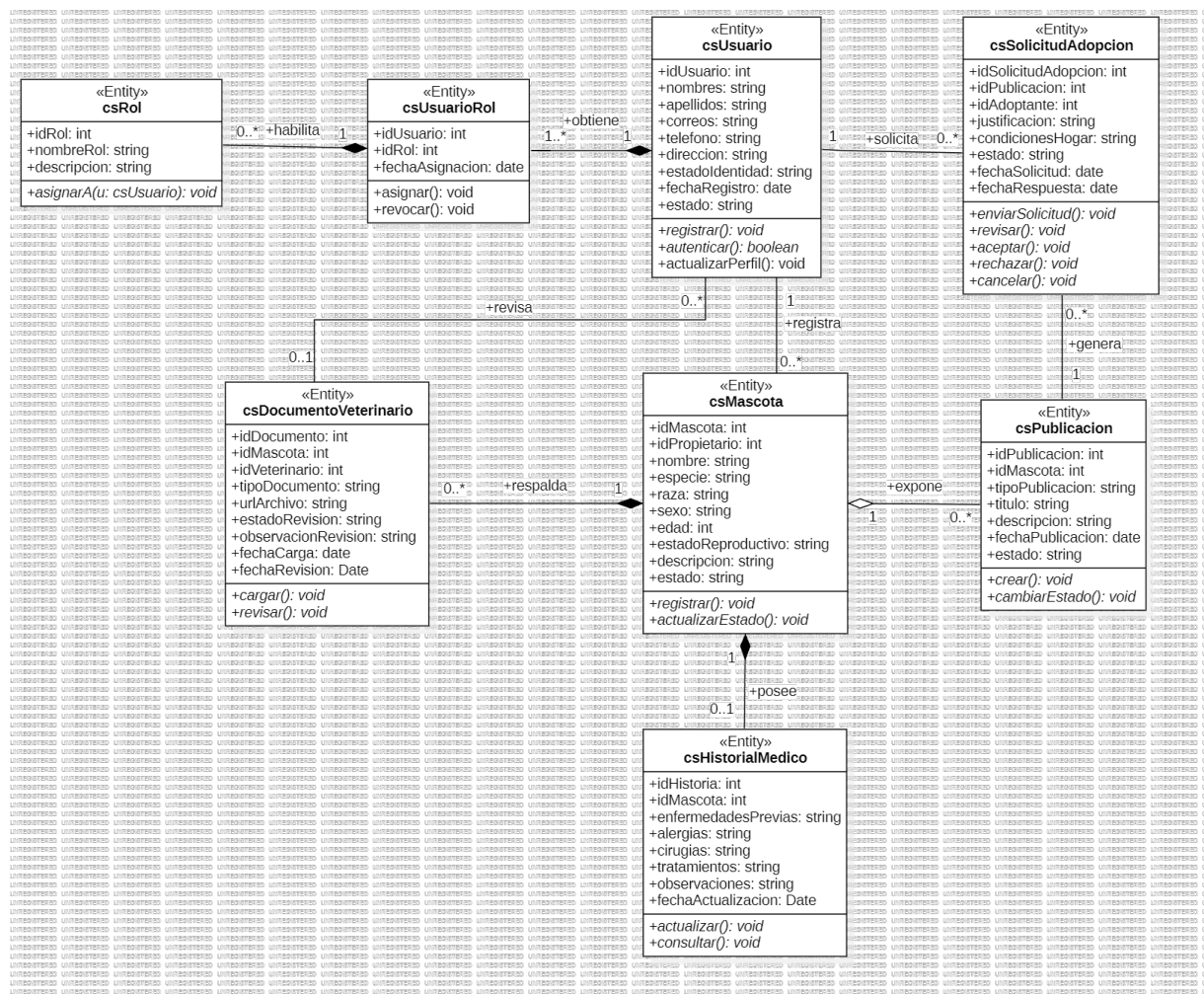


Figura 1: Diagrama de Clases del Módulo Gestión de Solicitud de Adopción “MundiPets”

La Figura 1 presenta el diagrama de clases del módulo de Gestión de Solicitud de Adopción, compuesto por ocho clases del dominio (csUsuario, csRol, csUsuarioRol, csMascota, csHistorialMedico, csPublicacion, csDocumentoVeterinario, csSolicitudAdopcion), estereotipadas como «Entity». El archivo nativo del modelo se encuentra en `Modelo/DiagramadeClases-GestionSolicitudAdopcion.mdj` dentro del repositorio del equipo.

5.2 Relaciones UML

Tabla 16: Relaciones UML del módulo

#	Relación	Cardinalidad	Etiqueta	Tipo
1	Usuario → Mascota	1 → 0..*	regista	Asociación
2	Usuario → UsuarioRol	1 → 1..*	obtiene	Composición
3	Rol → UsuarioRol	1 → 0..*	habilita	Composición
4	Mascota → HistorialMedico	1 → 0..1	posee	Composición
5	Mascota → Publicacion	1 → 0..*	expone	Agregación
6	Mascota → DocumentoVeterinario	1 → 0..*	respalda	Composición
7	Publicacion → SolicitudAdopcion	1 → 0..*	genera	Asociación
8	Usuario → SolicitudAdopcion	1 → 0..*	solicita	Asociación
9	Usuario → DocumentoVeterinario	0..* → 0..1	revisa	Asociación

Las composiciones reflejan clases cuya existencia depende de su contenedora (*UsuarioRol*, *HistorialMedico*, *DocumentoVeterinario*); la agregación Mascota–Publicacion refleja que la publicación conserva su propio ciclo de vida; el resto son asociaciones simples.

5.3 Estereotipos

Tabla 17: Estereotipo aplicado y su efecto en el código generado

Estereotipo	Clases	Efecto en el código C# generado
«Entity»	Las 8 clases del dominio	Ninguno –el generador no lo traduce–

El estereotipo «Entity» del *UML Standard Profile* se aplicó a las 8 clases del dominio como clasificación semántica dentro del modelo UML. La revisión del código generado (Sección 8) confirmó que la extensión *C# Code Generation and Reverse Engineering* no lo traduce a ninguna anotación, atributo o comentario en C#; su función es exclusivamente documental sobre el modelo y no interviene en el mecanismo de generación evaluado en este trabajo.

5.4 Diagramas complementarios

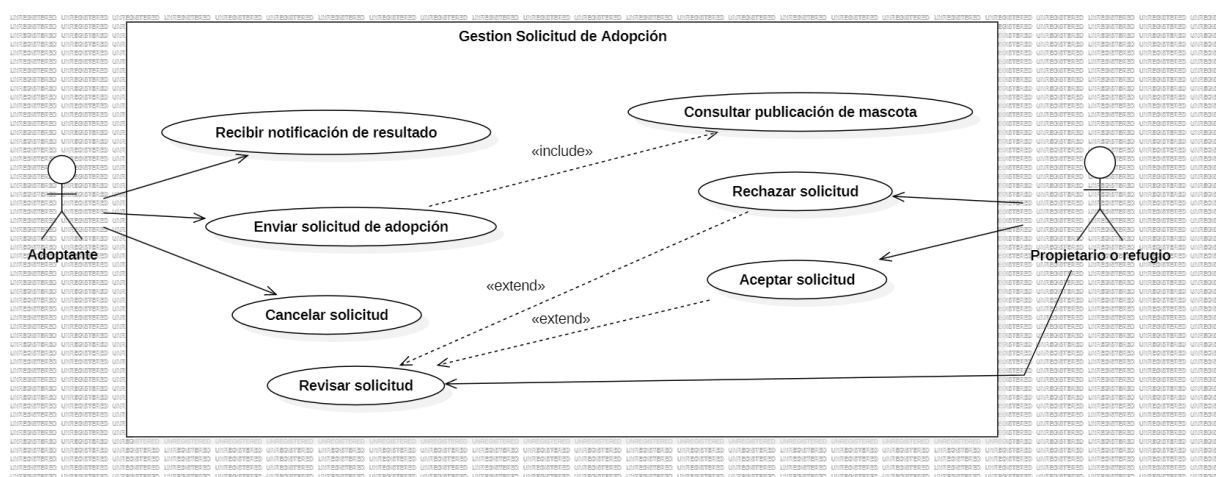


Figura 2: Diagrama Caso de Uso del Módulo Gestión de Solicitud de Adopción “MundiPets”

La Figura 2 presenta el diagrama de casos de uso del módulo de Gestión de Solicitud de Adopción, elaborado a partir del requisito funcional RF-05 y los actores identificados en la Sección 4.3. El adoptante consulta la publicación de una mascota disponible, envía la solicitud de adopción y recibe la notificación del resultado, pudiendo cancelarla mientras permanezca activa. El propietario o refugio revisa la solicitud recibida y decide entre aceptarla o rechazarla, ambos casos modelados como extensiones del caso de uso “Revisar solicitud”. Este diagrama complementa la vista estructural del diagrama de clases (Figura 1), aportando el contexto de interacción entre los actores y el sistema.

5.5 Validación del modelo en StarUML

La validación en StarUML es asíncrona: se ejecuta automáticamente cada vez que el modelo se guarda. Tras guardar el modelo final del módulo de Gestión de Solicitud de Adopción, el panel de resultados de StarUML confirmó “Validation Results — No errors”, sin advertencias sobre clases, atributos o relaciones, verificando la consistencia estructural del modelo previo a la generación de código (Figura 3).

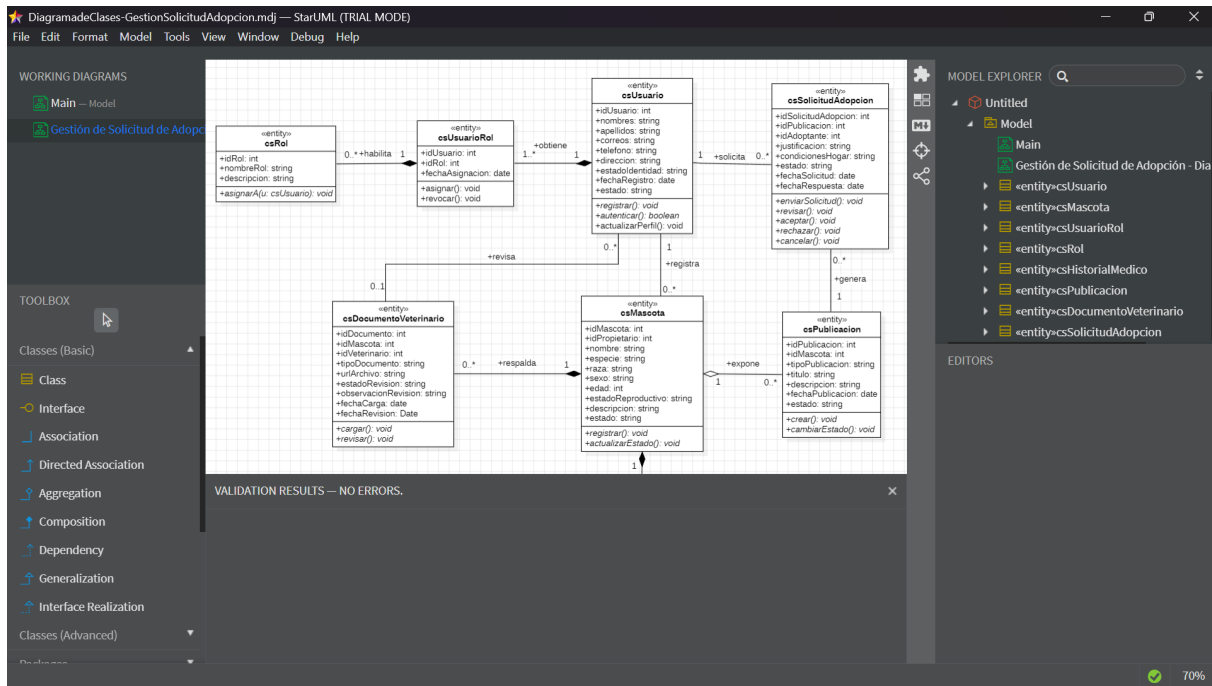


Figura 3: Resultado de la validación del modelo en StarUML — sin errores

6 Configuración del Generador

6.1 Plantilla y Parámetros

La herramienta seleccionada es StarUML, con el complemento C# Code Generation and Reverse Engineering, invocado desde la interfaz gráfica mediante Tools → C# → Generate Code (detalle de ejecución en la Sección 7). A diferencia de motores de plantillas editables (EJS, Velocity), la transformación está codificada de forma fija dentro del complemento (módulo `code-generator.js`), por lo que se clasifica como plantilla estándar, no personalizable desde la interfaz.

Tabla 18: Parámetros de configuración utilizados.

Parámetro	Valor para MundiPets
Herramienta / complemento	StarUML + C# Code Generation and Reverse Engineering
Elemento de origen	Model (8 clases del subsistema de gestión de solicitudes de adopción)
Lenguaje objetivo	C#
Tipo de plantilla	Estándar, fija en el código del complemento (no editable)
Ruta de salida — versión inicial	C:\Generado\MundiPets\VersionInicial
Ruta de salida — versión round-trip	C:\Generado\MundiPets\VersionRoundtrip (Sección 9)

6.2 Política de Regeneración y Decisiones de Mapeo

Política de regeneración. El complemento no ofrece regeneración selectiva ni bloques protegidos: cada ejecución regenera por completo el elemento del modelo seleccionado. Como estrategia propia, el equipo dirigió cada ejecución a una carpeta distinta (*VersionInicial* / *VersionRoundtrip*) para conservar evidencia de ambas versiones.

Comportamiento de tipos. El complemento no traduce nombres de tipo: si el atributo no está enlazado a un tipo de dato del modelo, copia literalmente el texto ingresado. Esto se confirmó en el código fuente del complemento (función `getType()` de `code-generator.js`) y coincide con los errores observados al compilar (Sección 8): *boolean*, *date* y *Date* se generaron tal cual y no son válidos en C#, por lo que se corrigieron manualmente a *bool* y *DateTime* en la copia de verificación.

Asociaciones. El diagrama de clases define una asociación real entre *csUsuario* y *csMascota* con multiplicidad 1 a muchos (un usuario puede tener varias mascotas). Sin embargo, la revisión del archivo generado *csMascota.cs* muestra que esa asociación no se tradujo mediante el mecanismo de colecciones del generador (bloque "(from associations)" de `writeClass()`, que produce campos `List<Tipo>/HashSet<Tipo>`): en su lugar, la clase contiene el campo suelto *idPropietario: int*, equivalente a una llave foránea manual. Esto indica que, aunque

la asociación existe en el diagrama, no quedó configurada como navegable desde el extremo correspondiente, por lo que el generador la trató como un atributo simple en vez de como relación de objetos. Se trata de una limitación de configuración del modelo, no del generador: el mecanismo existe (Tabla 19) pero no se activó para este caso.

Tabla 19: Decisiones de mapeo estructural.

Elemento UML	Salida en C#	Observación
Clase	class / abstract class	1 a 1 (ver Sección 7, trazabilidad)
Atributo	Campo con el mismo nombre y tipo tal cual fue escrito en el modelo	El tipo no se traduce automáticamente
Operación	Método con el mismo nombre y tipo de retorno declarado	Solo la firma; sin lógica de negocio (Sección 8)
Asociación navegable (mult. 1..*/0..*/*)	Campo <code>List<Tipo></code> u <code>HashSet<Tipo></code> en la clase de origen	Requiere que la asociación esté marcada como navegable en el modelo
Asociación <code>csUsuario-csMascota</code> (caso observado)	Campo suelto <code>idPropietario: int</code> en <code>csMascota</code> (sin <code>List<></code>)	La asociación existe en el diagrama pero no se generó como colección; ver detalle abajo

7 Proceso de generación

7.1 Ejecución y capturas

Para realizar la generación automática de código se abrió en StarUML el archivo nativo `DiagramadeClases-GestionSolicitudAdopcion.mdj`, correspondiente al subsistema de gestión de solicitudes de adopción de MundiPets. El modelo utilizado contiene ocho clases del dominio: `csUsuario`, `csMascota`, `csUsuarioRol`, `csRol`, `csHistorialMedico`, `csPublicacion`, `csDocumentoVeterinario` y `csSolicitudAdopcion`.

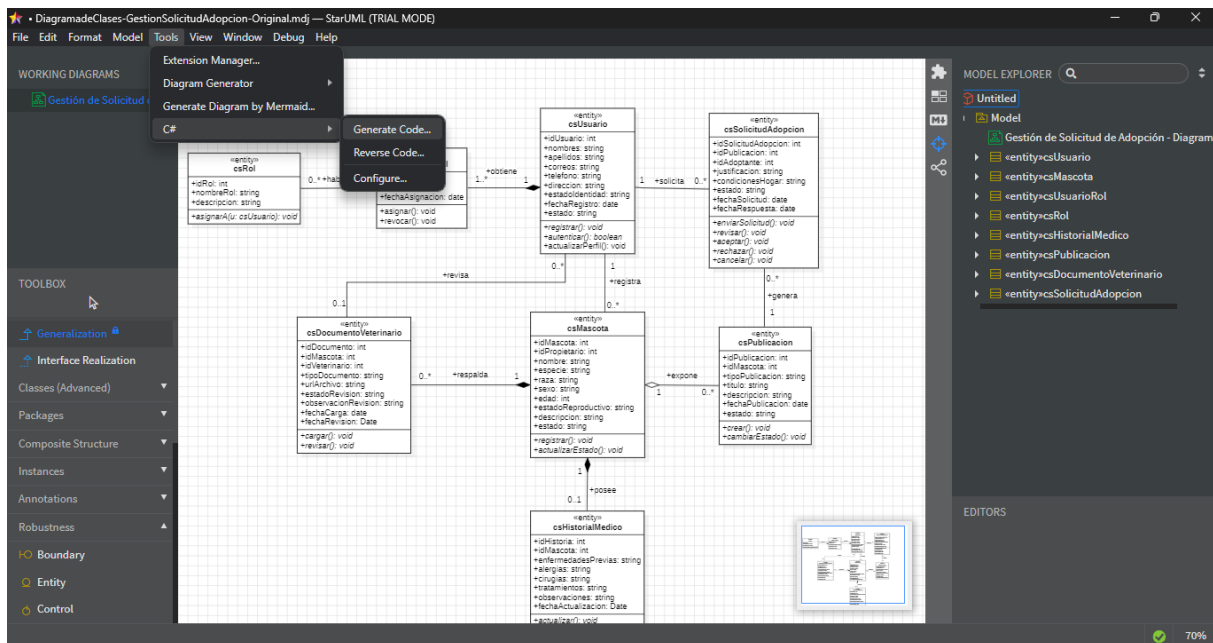


Figura 4: Invocación del generador de código C# desde StarUML.

La generación se ejecutó mediante la extensión C# Code Generation and Reverse Engineering, utilizando la ruta de menú `Tools, C#, Generate Code`. Como elemento base se seleccionó el modelo `Model`, que contiene las clases y relaciones del diagrama UML. La carpeta de salida configurada fue `VersionInicial`, ubicada en `C:\Generado\MundiPets`.

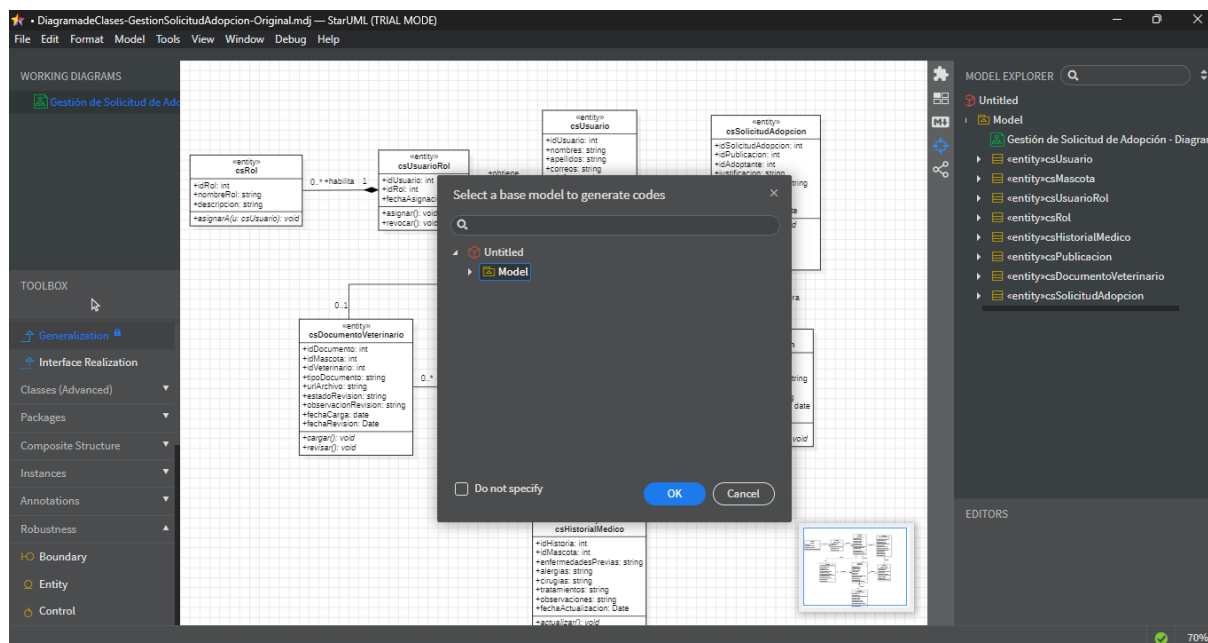


Figura 5: Selección del modelo base para la generación de código.

La extensión no presentó un panel de log detallado durante la ejecución. La finalización del proceso se verificó mediante la creación de los archivos C# en la carpeta de salida y la ausencia de mensajes de error.

El proceso inició aproximadamente a las 18:06 y tuvo una duración de 3 segundos, medida manualmente desde la confirmación de la generación hasta la creación de los archivos en la carpeta de salida.

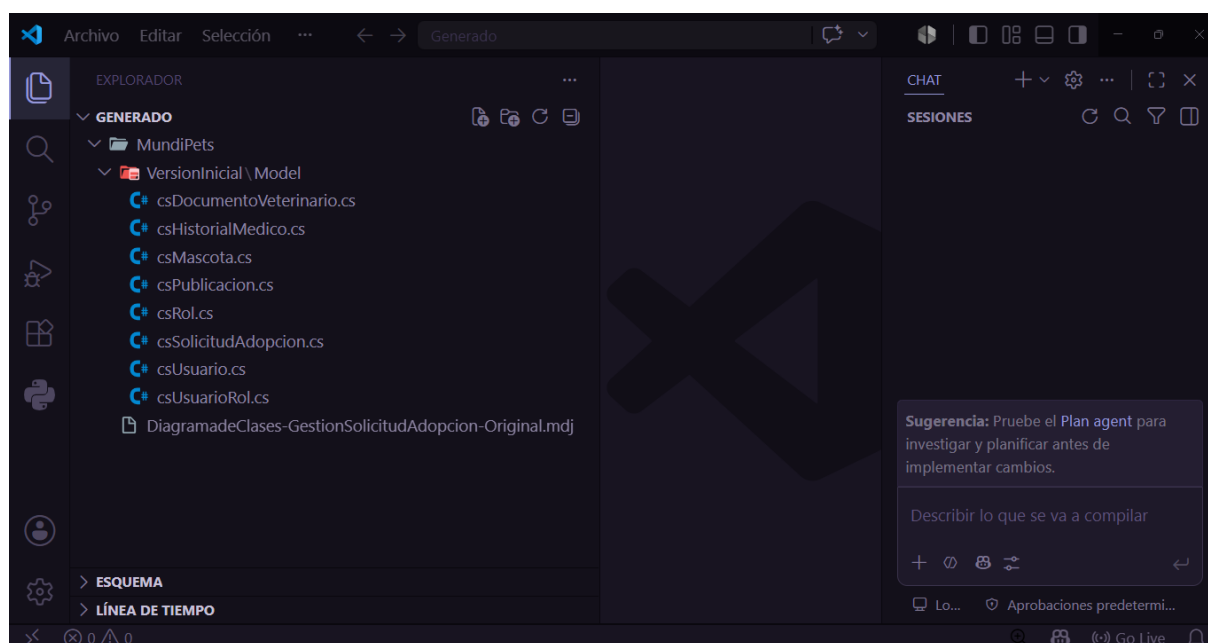


Figura 6: Archivos C# generados a partir del modelo UML de MundiPets.

La Figura 6 evidencia la creación automática de ocho archivos con extensión .cs. Cada archivo corresponde a una clase incluida en el modelo UML seleccionado, confirmando que la extensión completó el proceso de generación sin presentar mensajes de error.

7.2 Árbol del proyecto y trazabilidad clase-archivo

La estructura obtenida después de ejecutar el generador conserva una correspondencia directa entre las clases del modelo UML y los archivos de código fuente. Cada clase incluida dentro del modelo seleccionado produjo un

archivo independiente con extensión .cs, manteniendo el nombre definido en StarUML.

El código generado se almacenó en la carpeta C:\Generado\MundiPets\VersionInicial. El árbol resultante está compuesto por ocho archivos correspondientes a las ocho clases del subsistema de gestión de solicitudes de adopción.

Árbol del proyecto

```
VersionInicial\Model
csDocumentoVeterinario.cs
csHistorialMedico.cs
csMascota.cs
csPublicacion.cs
csRol.cs
csSolicitudAdopcion.cs
csUsuario.cs
csUsuarioRol.cs
```

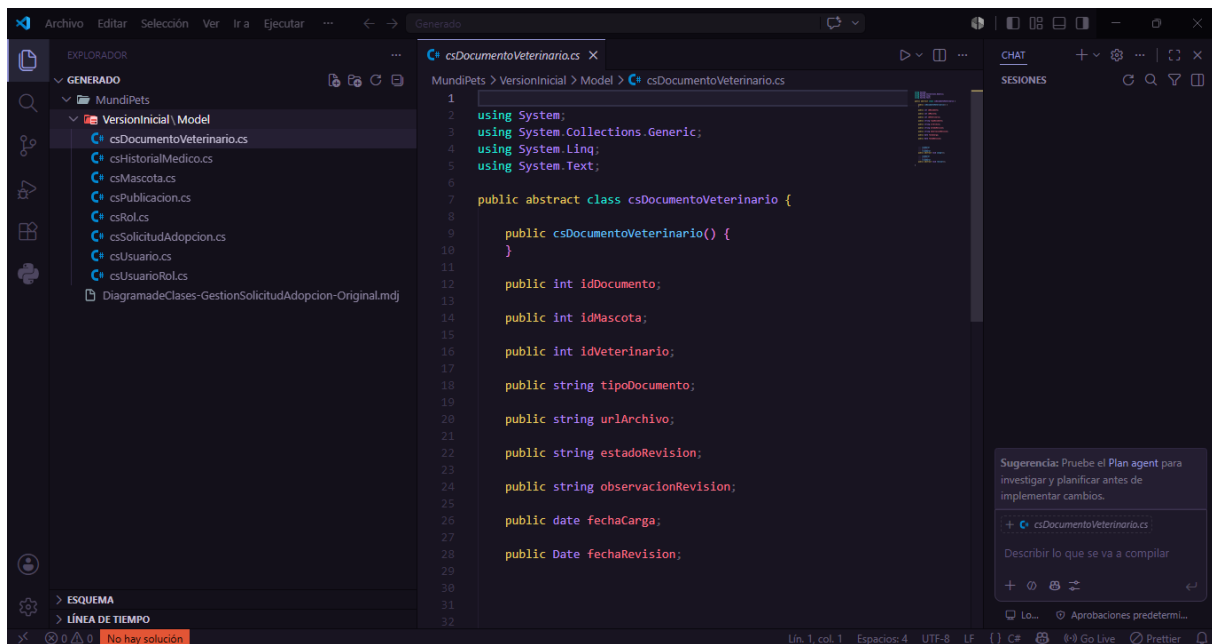


Figura 7: Árbol de archivos generado a partir de las clases UML del subsistema.

En la Figura 7 se observa que el generador creó un archivo de código fuente por cada clase UML del modelo. Esta organización facilita la identificación de los artefactos producidos y permite comprobar la trazabilidad desde el modelo hasta el código.

Tabla 20: Correspondencia entre clases UML y archivos C# generados.

Clase del modelo UML	Archivo C# generado	Correspondencia
csUsuario	csUsuario.cs	La clase UML produjo un archivo con el mismo nombre.
csMascota	csMascota.cs	Contiene los atributos y operaciones estructurales de la mascota.
csUsuarioRol	csUsuarioRol.cs	Representa la asignación de roles a los usuarios.
csRol	csRol.cs	Contiene la estructura correspondiente a los roles del sistema.
csHistorialMedico	csHistorialMedico.cs	Representa la información médica asociada a la mascota.
csPublicacion	csPublicacion.cs	Contiene la estructura de las publicaciones de mascotas.
csDocumentoVeterinario	csDocumentoVeterinario.cs	Representa los documentos veterinarios registrados.
csSolicitudAdopcion	csSolicitudAdopcion.cs	Contiene la estructura y operaciones de las solicitudes de adopción.

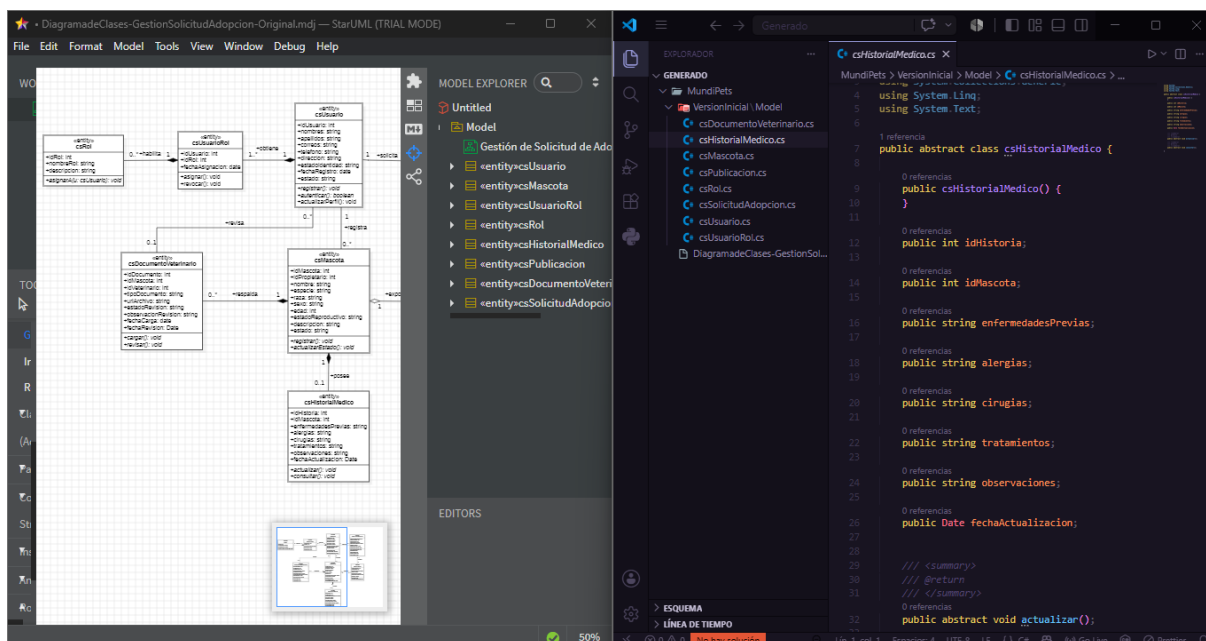


Figura 8: Trazabilidad entre las clases del modelo UML y los archivos C# generados.

La Figura 8 permite comparar visualmente las ocho clases presentes en el modelo UML con los ocho archivos obtenidos. La coincidencia en nombres y cantidad evidencia la transformación modelo a texto realizada por la extensión de StarUML.

8 Verificación del código generado

8.1 Compilación y ejecución

Para verificar la validez sintáctica del código generado por StarUML, los archivos C# se incorporaron en un proyecto de consola creado con el SDK de .NET, compilado desde la terminal integrada de Visual Studio Code mediante `dotnet build`. La primera compilación produjo nueve errores, originados porque el generador conservó literalmente los tipos `boolean`, `date` y `Date` del modelo UML, no válidos en C# (afectando a `csUsuario.cs`, `csUsuarioRol.cs`, `csPublicacion.cs`, `csDocumentoVeterinario.cs`, `csSolicitudAdopcion.cs` y `csHistorialMedico.cs`). Estos se corrigieron manualmente en la copia de verificación como `bool` y `DateTime`, conservando sin cambios los archivos originales generados por StarUML.

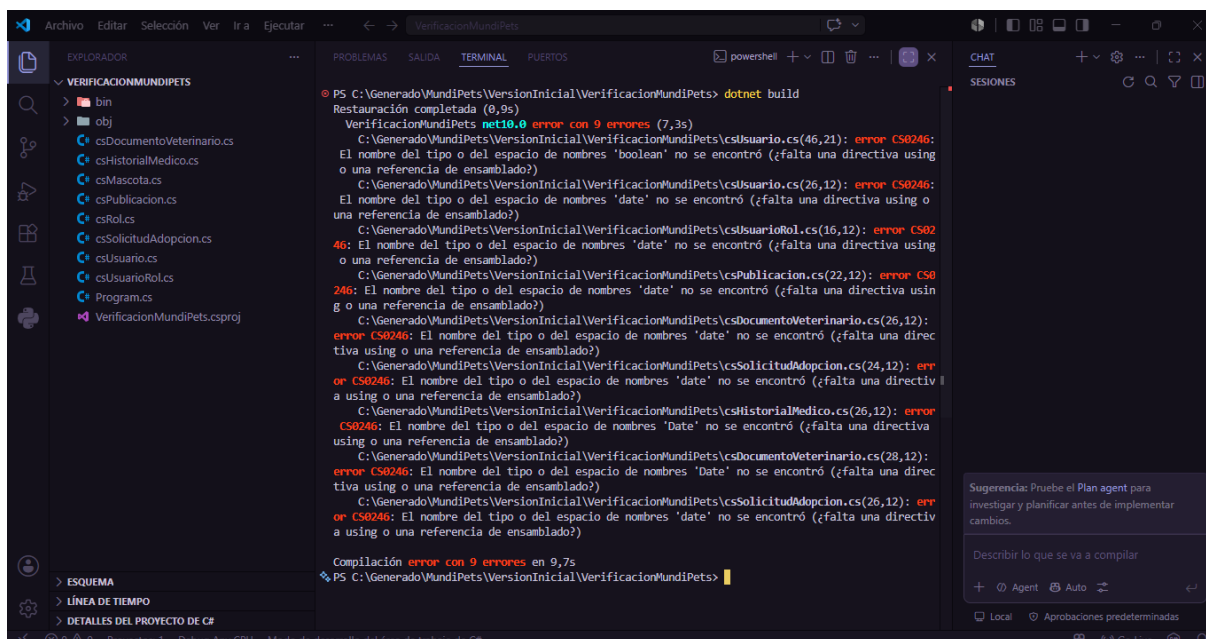


Figura 9: Primera compilación del código generado, con errores de incompatibilidad de tipos.

Tras corregir los tipos, la segunda compilación redujo los errores a tres, originados por los métodos `void asignar()` y `revocar()` (`csUsuarioRol`) y `actualizarPerfil()` (`csUsuario`), que contenían una instrucción `return` con expresión pese a no retornar valores; se eliminó dicha expresión.

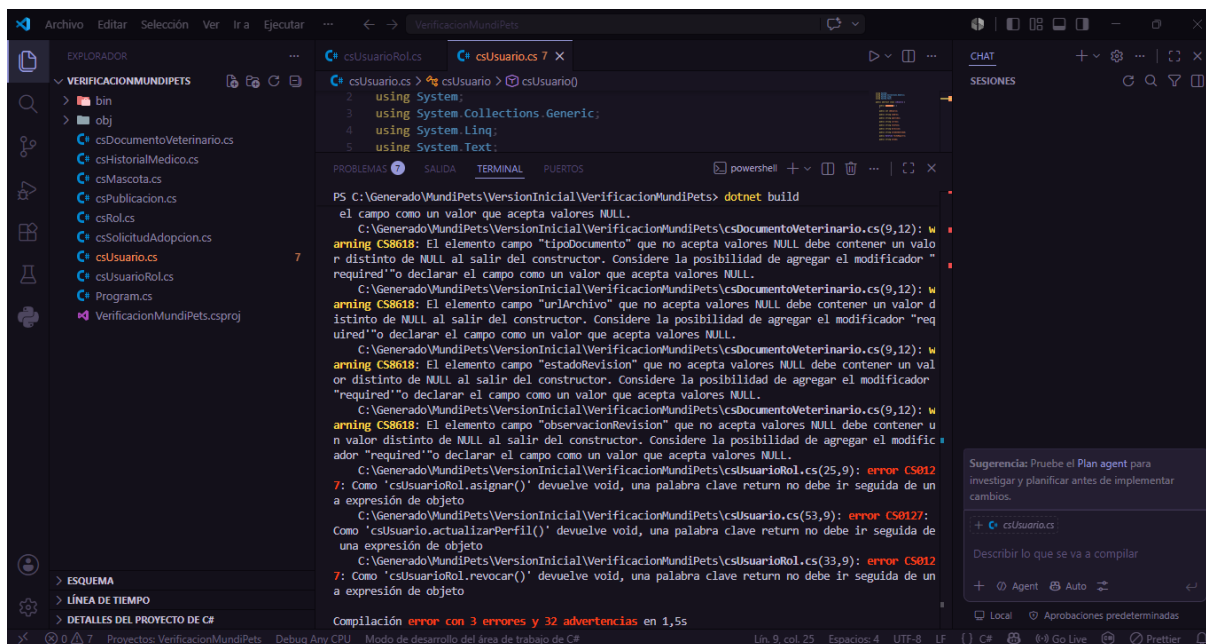


Figura 10: Segunda compilación después de adaptar los tipos UML al lenguaje C#.

Después de corregir los tipos y los retornos incorrectos, `dotnet build` finalizó con cero errores y treinta y dos advertencias, produciendo el ensamblado `VerificacionMundiPets.dll`. Las advertencias corresponden al código CS8618, relacionado con campos de tipo referencia no inicializados en los constructores generados; no impidieron la compilación ni la producción del ensamblado, pero evidencian que el generador crea principalmente la estructura de las clases sin configurar completamente las reglas modernas de nulabilidad de C#.

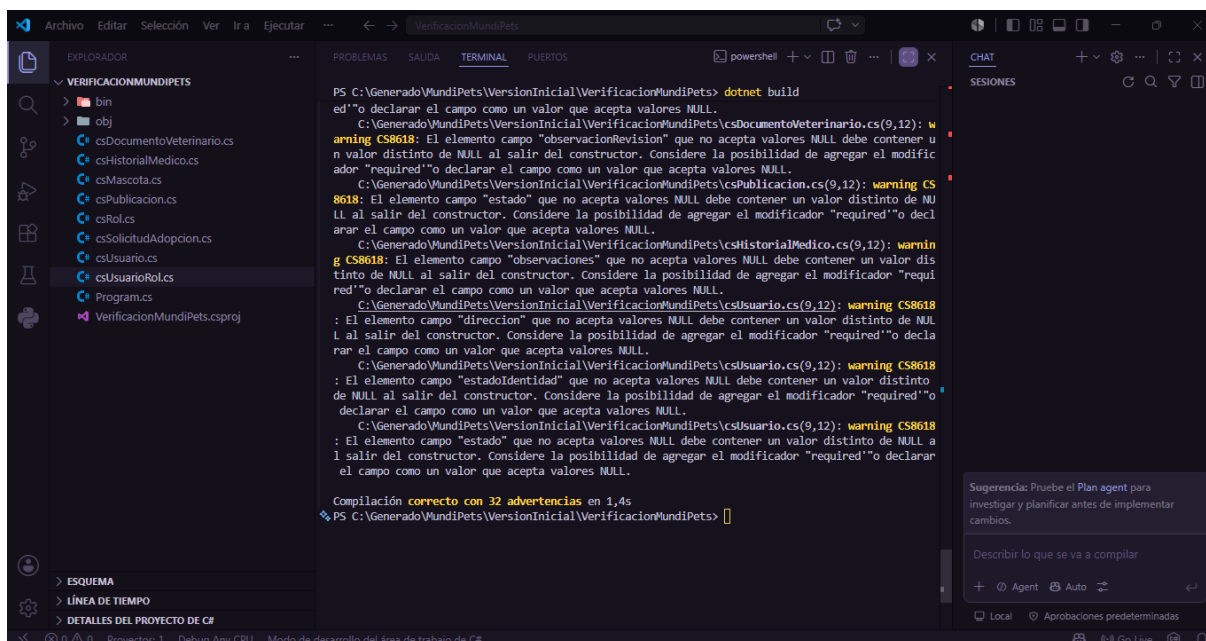


Figura 11: Compilación exitosa del código generado después de aplicar correcciones de compatibilidad.

Con el proyecto compilado, se ejecutó una prueba mínima (`dotnet run`) mediante un archivo `Program.cs` creado manualmente y mantenido separado de los archivos generados. La prueba cargó por reflexión la clase `csSolicitudAdopcion`, reconociendo sus 8 campos (`idSolicitudAdopcion`, `idPublicacion`, `idAdoptante`, `justificacion`, `condicionesHogar`, `estado`, `fechaSolicitud`, `fechaRespuesta`) y sus 5 métodos (`enviarSolicitud`, `revisar`, `aceptar`, `rechazar`, `cancelar`). Al ser una clase abstracta no se instanció directamente, pero la carga del tipo confirmó su correcta incorporación al ensamblado.

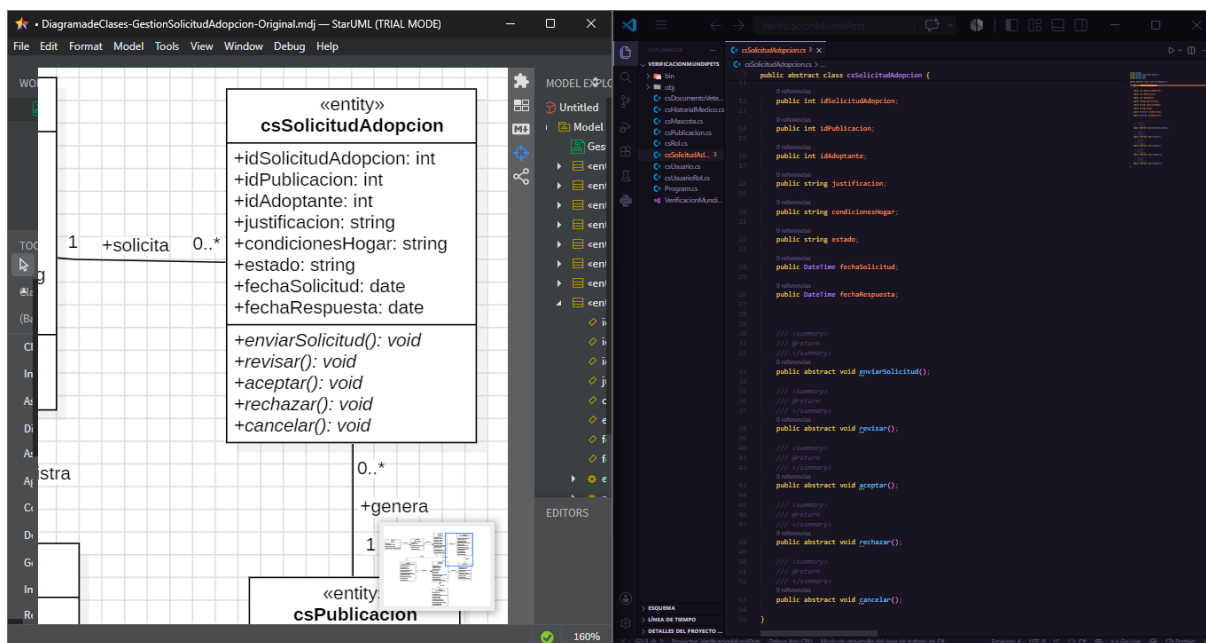


Figura 12: Ejecución de una prueba estructural sobre la clase `csSolicitudAdopcion`.

Tabla 21: Resultados de la compilación y ejecución del código generado.

Prueba	Resultado real	Evidencia
Primera compilación	9 errores de tipos incompatibles	Figura 9
Segunda compilación	3 errores y 32 advertencias	Figura 10
Compilación final	0 errores y 32 advertencias	Figura 11
Ejecución mínima	Clase cargada; 8 campos y 5 métodos reconocidos	Figura 12

8.2 Correspondencia modelo-código

Para comprobar la trazabilidad detallada entre el modelo y el código, se comparó la clase `csSolicitudAdopcion` definida en StarUML con el archivo `csSolicitudAdopcion.cs` producido por el generador, considerando el nombre de la clase, sus atributos, los tipos de datos y las operaciones transformadas en métodos. Los atributos `idSolicitudAdopcion`, `idPublicacion`, `idAdoptante`, `justificacion`, `condicionesHogar` y `estado` conservaron sus nombres y tipos estructurales; `fechaSolicitud` y `fechaRespuesta`, originalmente `date`, se adaptaron manualmente a `DateTime`. Las operaciones `enviarSolicitud()`, `revisar()`, `aceptar()`, `rechazar()` y `cancelar()` se generaron como métodos abstractos `void`, únicamente con su firma, sin la lógica de negocio correspondiente (Tabla 22).

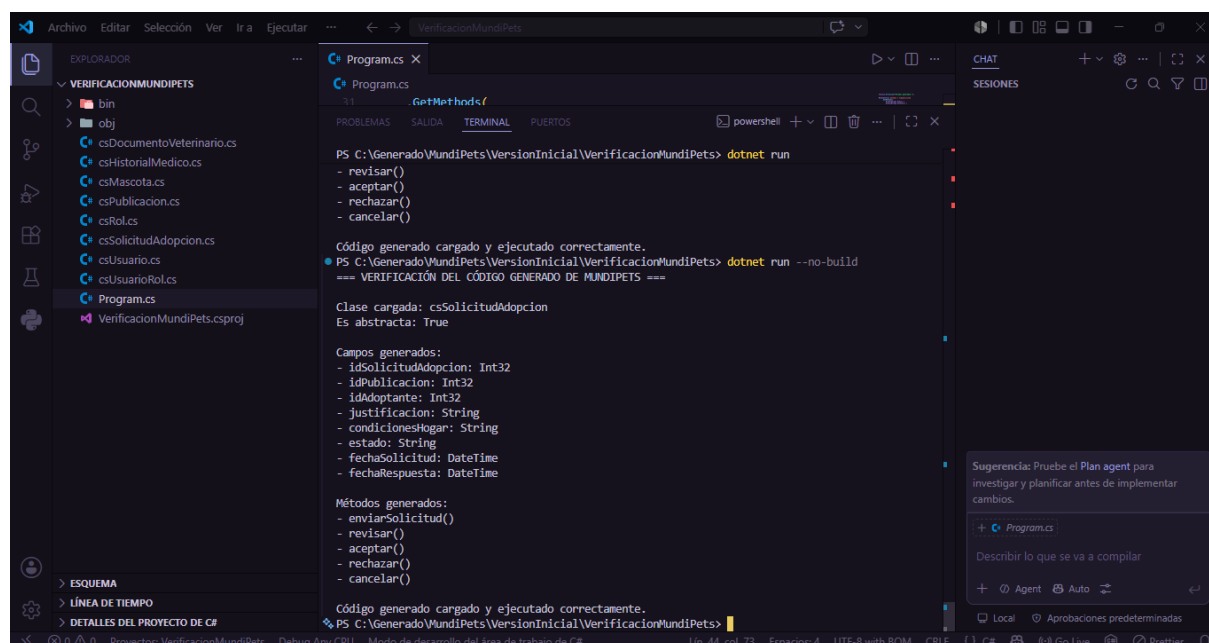


Figura 13: Correspondencia entre la clase UML `csSolicitudAdopcion` y el archivo C# generado.

Tabla 22: Correspondencia entre la clase UML `csSolicitudAdopcion` y el código generado.

Elemento UML	Código C# generado	Resultado
Clase <code>csSolicitudAdopcion</code>	<code>abstract class csSolicitudAdopcion</code>	Correspondencia directa
<code>idSolicitudAdopcion: int</code>	<code>int idSolicitudAdopcion</code>	Conservado
<code>idPublicacion: int</code>	<code>int idPublicacion</code>	Conservado
<code>idAdoptante: int</code>	<code>int idAdoptante</code>	Conservado
<code>justificacion: string</code>	<code>string justificacion</code>	Conservado
<code>condicionesHogar: string</code>	<code>string condicionesHogar</code>	Conservado
<code>estado: string</code>	<code>string estado</code>	Conservado
<code>fechaSolicitud: date</code>	<code>DateTime fechaSolicitud</code>	Adaptado manualmente
<code>fechaRespuesta: date</code>	<code>DateTime fechaRespuesta</code>	Adaptado manualmente
<code>enviarSolicitud(): void</code>	<code>abstract void enviarSolicitud()</code>	Firma generada
<code>revisar(): void</code>	<code>abstract void revisar()</code>	Firma generada
<code>aceptar(): void</code>	<code>abstract void aceptar()</code>	Firma generada
<code>rechazar(): void</code>	<code>abstract void rechazar()</code>	Firma generada
<code>cancelar(): void</code>	<code>abstract void cancelar()</code>	Firma generada

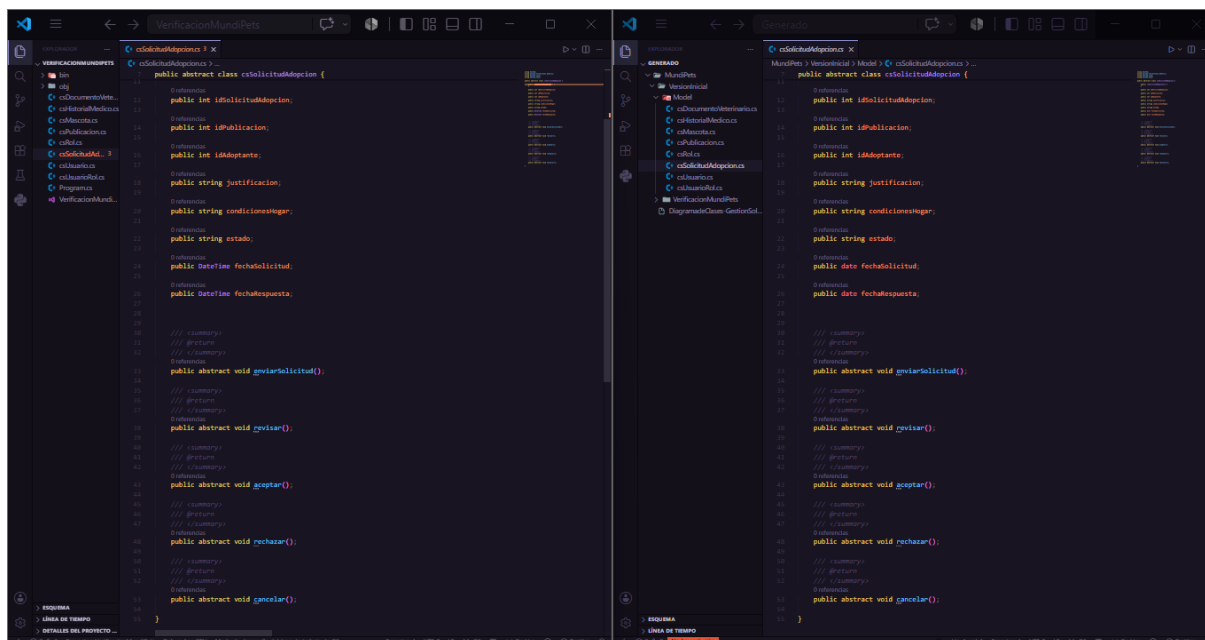
8.3 Limitaciones detectadas

Durante la compilación y ejecución del código generado se identificaron limitaciones relacionadas con la compatibilidad de tipos, la generación de métodos, la inicialización de campos y la ausencia de lógica de negocio. Estas observaciones permiten distinguir claramente los elementos producidos automáticamente por StarUML de las correcciones y componentes añadidos manualmente para realizar la verificación (Tabla 23).

Tabla 23: Limitaciones identificadas en el código generado por StarUML.

Limitación detectada	Evidencia observada	Corrección aplicada	Tipo de intervención
Tipos UML incompatibles con C#	boolean, date y Date produjeron 9 errores CS0246	boolean→bool; date/Date→DateTime	Manual
Retornos incorrectos en métodos void	asignar(), revocar(), actualizarPerfil() con return + expresión	Se eliminó la expresión de retorno	Manual
Campos string sin inicializar	32 advertencias CS8618 en la compilación final	Se documentaron; no impidieron compilar/ejecutar	No corregido
Generación de clases abstractas	csSolicitudAdopcion generada como clase abstracta	Prueba mediante reflexión, sin instanciación directa	Código de prueba manual
Métodos sin lógica de negocio	Firmas de enviarSolicitud(), revisar(), aceptar(), rechazar(), cancelar()	Implementación pendiente para el desarrollo del PFC	Pendiente
Ausencia de persistencia y CRUD completo	Sin conexión a BD, repositorios ni operaciones CRUD	Se desarrollará posteriormente	Pendiente
Ausencia de validaciones	Sin reglas de estado, campos obligatorios o permisos	Se implementarán manualmente	Pendiente

La herramienta generó las clases, campos y firmas de los métodos, pero no produjo la lógica de negocio necesaria para gestionar solicitudes de adopción. Tampoco generó interfaces, persistencia, repositorios, validaciones ni operaciones CRUD completas. Por tanto, el resultado constituye una base estructural del sistema y no una aplicación terminada.

**Figura 14:** Comparación entre el código original generado y la corrección manual de tipos.

StarUML generó los campos `fechaSolicitud` y `fechaRespuesta` con el tipo `date`; en la copia de verificación ambos fueron sustituidos manualmente por `DateTime`, permitiendo que el archivo fuera reconocido por el compilador de C#.

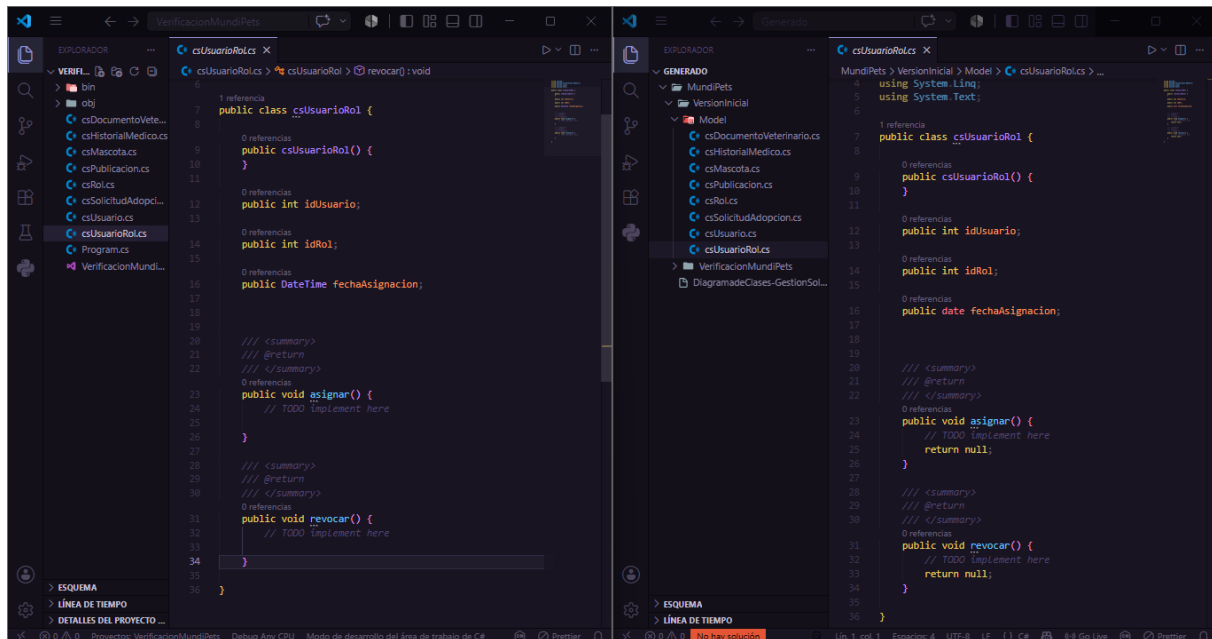


Figura 15: Corrección manual de expresiones de retorno incompatibles con métodos void.

Los métodos `asignar()` y `revocar()` fueron generados con la instrucción `return null`, pese a estar declarados con tipo de retorno `void`. En la copia de verificación se eliminaron las instrucciones `return null`, conservando las firmas y el espacio destinado a la implementación posterior de la lógica de negocio.

9 Cierre del ciclo: round-trip engineering

9.1 Cambio aplicado al modelo

Para comprobar el cierre controlado del ciclo modelo-código se aplicó un cambio significativo sobre la clase `csSolicitudAdopcion`. Antes de la modificación, la clase permitía cancelar una solicitud mediante la operación `cancelar(): void`, pero no contenía información que permitiera registrar la causa de la cancelación.

Como mejora del modelo, se añadió el atributo público `motivoCancelacion: string` y se modificó la operación de cancelación para recibir el parámetro `motivo: string`. La nueva firma quedó definida como `cancelar(motivo: string): void`. Este cambio permite representar explícitamente la razón proporcionada por el adoptante al cancelar una solicitud.

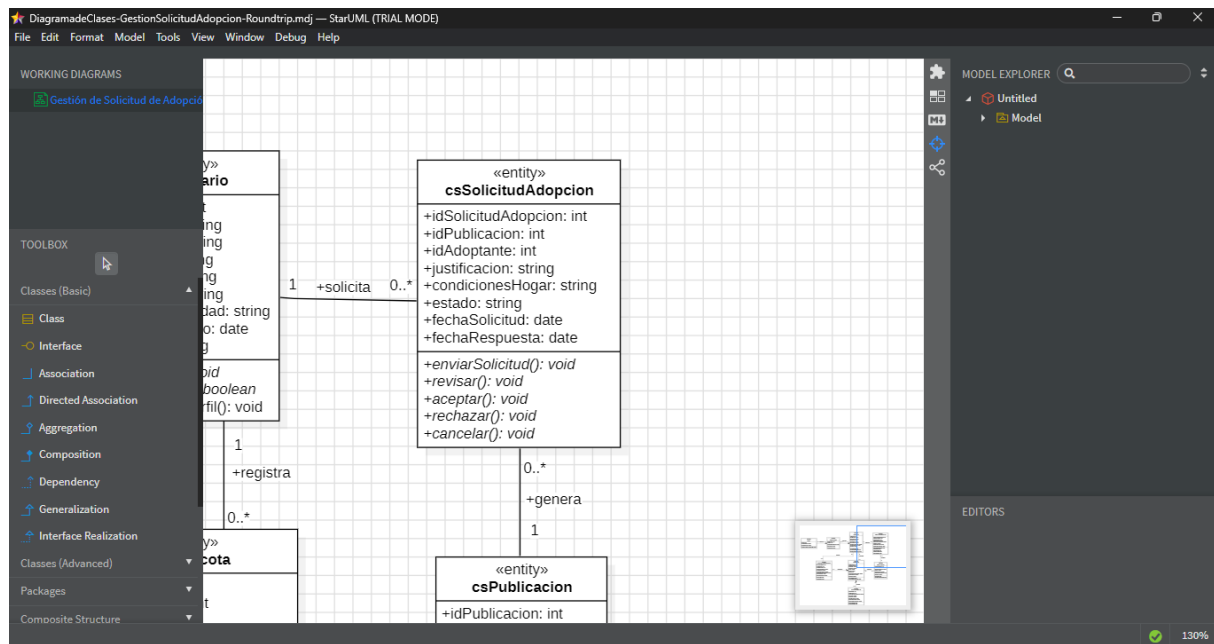


Figura 16: Clase csSolicitudAdopcion antes de aplicar el cambio de round-trip engineering.

```

1  public abstract class csSolicitudAdopcion {
2
3      // atributos
4      public int idSolicitudAdopcion;
5
6      // atributos
7      public int idPublicacion;
8
9      // atributos
10     public int idAdoptante;
11
12     // atributos
13     public string justificacion;
14
15     // atributos
16     public string condicionesHogar;
17
18     // atributos
19     public string estado;
20
21     // atributos
22     public date fechaSolicitud;
23
24     // atributos
25     public date fechaRespuesta;
26
27
28     // summary
29     /// 
30     ///   

31     /// 
32     ///   

33     ///   

34     public abstract void enviarSolicitud();
35
36     // summary
37     /// 
38     ///   

39     /// 
40     ///   

41     ///   

42     public abstract void revisar();
43
44     // summary
45     /// 
46     ///   

47     /// 
48     ///   

49     ///   

50     public abstract void aceptar();
51
52     // summary
53     /// 
54     ///   

55     /// 
56     ///   

57     ///   

58     public abstract void rechazar();
59
60     // summary
61     /// 
62     ///   

63     /// 
64     ///   

65     ///   

66     public abstract void cancelar();
67
68 }

```

Figura 17: Código de csSolicitudAdopcion antes de la regeneración.

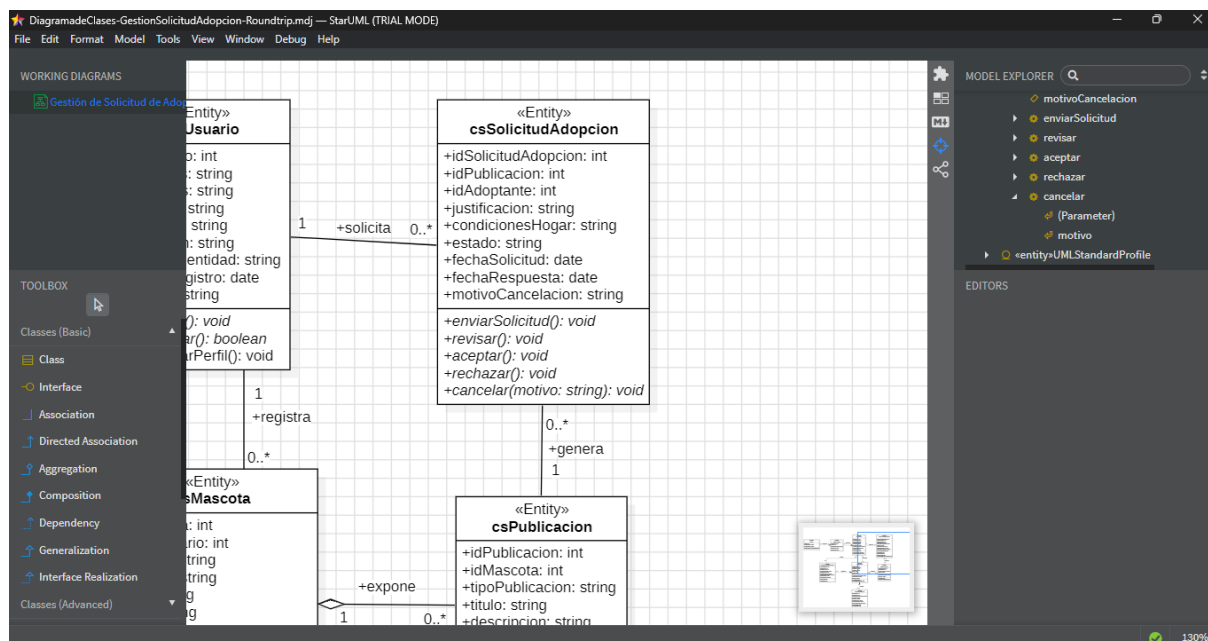


Figura 18: Clase csSolicitudAdopcion después de incorporar el motivo de cancelación.

Las Figuras 16 y 18 muestran el estado de la clase antes y después de la modificación. El cambio afecta tanto la estructura de datos como la operación relacionada con la cancelación, por lo que constituye una modificación funcionalmente relevante para el subsistema de gestión de adopciones.

9.2 Evidencia antes y después

Una vez guardado el cambio en el modelo UML, se ejecutó nuevamente la extensión C# Code Generation and Reverse Engineering. Para preservar la evidencia inicial, el código regenerado se almacenó en la carpeta VersionRoundtrip, ubicada en C:\Generado\MundiPets.

```

1  namespace MundiPets
2  {
3      public abstract class csSolicitudAdopcion
4      {
5          // Propiedades
6          public int idSolicitudAdopcion;
7          public int idPublicacion;
8          public int idAdoptante;
9          public string justificacion;
10         public string condicionesHogar;
11         public string estado;
12         public date fechaSolicitud;
13         public date fechaRespuesta;
14         public string motivoCancelacion;
15         // Métodos
16         public abstract void enviarSolicitud();
17         public abstract void revisar();
18         public abstract void aceptar();
19         public abstract void rechazar();
20         public abstract void cancelar(string motivo);
21     }
22 }

```

Figura 19: Código de csSolicitudAdopcion después de regenerar el modelo modificado.

La regeneración produjo una nueva versión del archivo csSolicitudAdopcion.cs. En esta versión se incorporó el campo motivoCancelacion de tipo string y la operación cancelar() fue actualizada para recibir el parámetro motivo de tipo string.

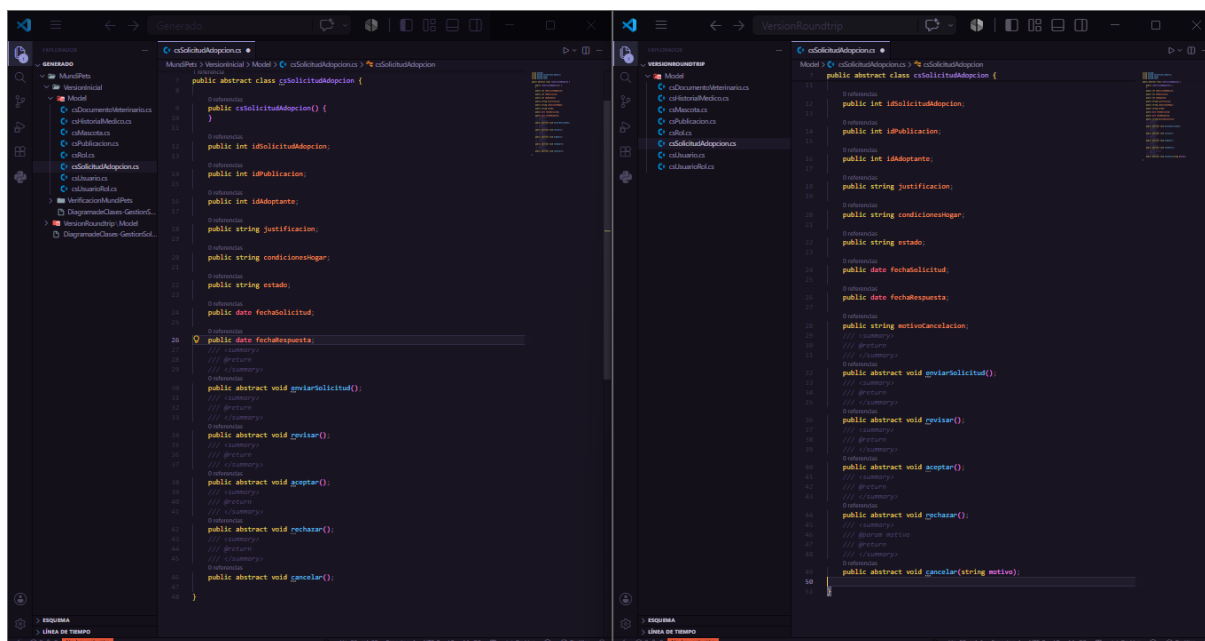


Figura 20: Comparación del código antes y después de la regeneración.

La Figura 20 permite comprobar que el archivo anterior no contenía el atributo `motivoCancelacion` y presentaba la operación `cancelar()` sin parámetros. En la versión regenerada ambos cambios aparecen de acuerdo con la modificación realizada en StarUML, demostrando la trazabilidad en dirección modelo a código.

Tabla 24: Comparación de los elementos antes y después de la regeneración.

Elemento	Antes del cambio	Después del cambio
Atributo motivoCancelacion	No existía	Generado como campo string
Operación de cancelación	cancelar(): void	cancelar(motivo: string): void
Archivo afectado	csSolicitudAdopcion.cs	csSolicitudAdopcion.cs actualizado
Dirección comprobada	Modelo a código inicial	Modelo modificado a código regenerado

La prueba realizada comprobó el cierre del ciclo en dirección modelo a código. No se ejecutó una sincronización completa código a modelo, debido a que el objetivo de la demostración fue verificar la propagación controlada de cambios desde StarUML hacia los archivos C# generados.

10 Reparto del trabajo

El trabajo se distribuyó de forma equitativa entre los tres integrantes del Equipo B. La Tabla 25 detalla cada sección del documento y su responsable.

Tabla 25: Reparto del trabajo por sección.

Sección	Desarrollado por
1. Resumen	Gutierrez Genesis
2. Marco teórico	Cordova Mayra
3. Justificación de la selección de StarUML	Gutierrez Genesis
4. Descripción del subsistema del PFC modelado	Gutierrez Genesis
5. Modelo UML de origen	Gutierrez Genesis
6. Configuración del generador	Cordova Mayra
7. Proceso de generación	Naranjo Anderson
8. Verificación del código generado	Naranjo Anderson
9. Cierre del ciclo: round-trip engineering	Naranjo Anderson
10. Reparto del trabajo	Gutierrez Genesis
11. Conclusiones	Gutierrez Genesis
13. Anexos	Gutierrez Genesis

La evidencia objetiva de esta distribución se encuentra en el historial de *commits* del repositorio GitHub, donde cada integrante contribuyó con su propia identidad Git. El registro de aportaciones puede consultarse en la pestaña Insights → Contributors del repositorio, cuya captura se incluye en Evidencias/Capturas/.

Como respaldo de la distribución declarada en la Tabla 25, se presenta la evidencia extraída del repositorio GitHub del equipo. La Figura 21 muestra la pestaña *Insights* → *Contributors*, donde se refleja la contribución individual de cada integrante mediante su propia identidad Git a lo largo del desarrollo del proyecto.

Figura 21: Contribuciones por integrante en GitHub (*Insights* → *Contributors*).

11 Conclusiones

La experiencia de aplicar *Model-Driven Development* sobre el módulo de Gestión de Solicitud de Adopción de MundiPets permitió comprobar, en un caso real y acotado, que StarUML es capaz de completar el ciclo básico de generación automática de código a partir de un diagrama de clases validado. Las ocho clases del subsistema se transformaron en igual número de archivos C# en cuestión de segundos, conservando de forma directa nombres de clases, atributos y firmas de operaciones, lo que confirma el valor de la trazabilidad modelo-código como principio central de MDD.

Sin embargo, el ejercicio también evidenció que el código generado constituye únicamente una base estructural y no una aplicación funcional. La herramienta no produjo lógica de negocio, persistencia, validaciones ni operaciones CRUD completas, y presentó limitaciones concretas de traducción de tipos (*date*, *boolean*) que exigieron corrección manual para lograr una compilación exitosa. De igual manera, se identificó que una asociación navegable del modelo no se generó como colección, lo que reforzó la importancia de configurar correctamente la navegabilidad de las relaciones antes de ejecutar el generador, más que un defecto atribuible a la herramienta en sí.

El cierre del ciclo mediante *round-trip engineering* —incorporar el atributo *motivoCancelacion* y el parámetro correspondiente en la operación *cancelar()*— permitió verificar de manera práctica que los cambios realizados en el modelo se propagan de forma consistente hacia el código regenerado, validando la trazabilidad en la dirección modelo → código dentro del alcance probado por el equipo.

En términos de aprendizaje, el equipo concluye que el valor real de MDD para un proyecto como el PFC no reside en obtener una aplicación terminada de manera automática, sino en acelerar la fase inicial de construcción, reducir errores de transcripción entre el diseño y la implementación, y obligar a que el modelo UML sea preciso y esté libre de errores antes de generar artefactos. Asimismo, se identificó que la elección de la herramienta (StarUML frente a Visual Paradigm) debe responder al alcance real del proyecto y no necesariamente a la herramienta con más funcionalidades disponibles, dado que la simplicidad de configuración resultó más determinante que la madurez de las plantillas ORM para un módulo de ocho clases.

Como limitación general del ejercicio, se reconoce que la generación fue evaluada únicamente en dirección modelo → código y sobre un módulo acotado del sistema, por lo que no es posible extrapolar directamente estos resultados a la totalidad del PFC ni a escenarios con sincronización bidireccional completa.

12 Referencias

- [1] Antonio Bucchiarone et al. «Grand challenges in model-driven engineering: an analysis of the state of the research». En: *Software and Systems Modeling* 19.1 (2020), págs. 5-13. DOI: 10.1007/s10270-019-00773-6. URL: <https://doi.org/10.1007/s10270-019-00773-6>.
- [2] Zeineb Zammel et al. «Towards AI-Enabled Model-Driven Architecture: Systematic Literature Review». En: *Proceedings of the 17th International Conference on Agents and Artificial Intelligence*. SCITEPRESS - Science y Technology Publications, 2025, págs. 1380-1387. DOI: 10.5220/0013374000003890. URL: <https://doi.org/10.5220/0013374000003890>.
- [3] Álvaro Domingo et al. «Evaluating the Benefits of Model-Driven Development». En: 2020, págs. 353-367. DOI: 10.1007/978-3-030-49435-3_22. URL: https://doi.org/10.1007/978-3-030-49435-3_22.
- [4] Javier Troya et al. «Model Transformation Testing and Debugging: A Survey». En: *ACM Computing Surveys* 55.4 (2023), págs. 1-39. DOI: 10.1145/3523056. URL: <https://doi.org/10.1145/3523056>.
- [5] Abeer Zafar et al. «Exploring the Effectiveness and Trends of Domain-Specific Model Driven Engineering: A Systematic Literature Review (SLR)». En: *IEEE Access* 12 (2024), págs. 86809-86830. DOI: 10.1109/ACCESS.2024.3414503. URL: <https://doi.org/10.1109/ACCESS.2024.3414503>.
- [6] Achraf Abouzahra, Abdelafou Sabraoui y Karim Afdel. «Model composition in Model Driven Engineering: A systematic literature review». En: *Information and Software Technology* 125 (2020), pág. 106316. DOI: 10.1016/j.infsof.2020.106316. URL: <https://doi.org/10.1016/j.infsof.2020.106316>.
- [7] Gilles Vanwormhoudt et al. «Template based model engineering in UML». En: *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems*. New York, NY, USA: ACM, oct. de 2020, págs. 47-56. DOI: 10.1145/3365438.3410988. URL: <https://doi.org/10.1145/3365438.3410988>.
- [8] Xin He y Tao Zan. «BIT: A template-based approach to incremental and bidirectional model-to-text transformation». En: *Journal of Systems and Software* 216 (2024), pág. 112148. DOI: 10.1016/j.jss.2024.112148. URL: <https://doi.org/10.1016/j.jss.2024.112148>.
- [9] Daniel Rosca y Luis Domingues. «A Systematic Comparison of Roundtrip Software Engineering Approaches applied to UML Class Diagram». En: *Procedia Computer Science* 181 (2021), págs. 861-868. DOI: 10.1016/j.procs.2021.01.240. URL: <https://doi.org/10.1016/j.procs.2021.01.240>.
- [10] Sultan Almutairi, Athanasios Zolotas y Dimitris Kolovos. «Towards round-Trip engineering of code fragments embedded in models». En: *Proceedings - ACM/IEEE 25th International Conference on Model Driven Engineering Languages and Systems, MODELS 2022: Companion Proceedings*. Association for Computing Machinery, Inc, oct. de 2022, págs. 529-538. DOI: 10.1145/3550356.3561578. URL: <https://doi.org/10.1145/3550356.3561578>.
- [11] Yong-Hyuk Kim. «Automatic Code Model Generation for State Machine with Internal Transitions». En: *Journal of the Korea Institute of Information and Communication Engineering* 28.7 (2024), págs. 841-849. DOI: 10.6109/jkiice.2024.28.7.841. URL: <https://doi.org/10.6109/jkiice.2024.28.7.841>.
- [12] José Antonio Hernández López, José Luis Cánovas Izquierdo y Jesús Sánchez Cuadrado. «ModelSet: a dataset for machine learning in model-driven engineering». En: *Software and Systems Modeling* 21.3 (2022), págs. 967-986. DOI: 10.1007/s10270-021-00929-3. URL: <https://doi.org/10.1007/s10270-021-00929-3>.
- [13] Ana C. Marcén et al. «A Systematic Literature Review of Model-Driven Engineering Using Machine Learning». En: *IEEE Transactions on Software Engineering* 50.9 (2024), págs. 2269-2293. DOI: 10.1109/TSE.2024.3430514. URL: <https://doi.org/10.1109/TSE.2024.3430514>.
- [14] Fatemeh Chitforoush, Maryam Yazdandoost y Raman Ramsin. «Methodology Support for the Model Driven Architecture». En: *14th Asia-Pacific Software Engineering Conference (APSEC 2007)*. IEEE, 2007, págs. 454-461. DOI: 10.1109/APSEC.2007.69. URL: <https://doi.org/10.1109/APSEC.2007.69>.
- [15] Julia N. Korongo, Samuel T. Mbugua y Samuel M. Mbuguah. «A Review Paper on Application of Model-Driven Architecture in Use-Case Driven Pervasive Software Development». En: *International Journal of Computer Trends and Technology* 70.3 (2022), págs. 19-26. DOI: 10.14445/22312803/IJCTT-V70I3P104. URL: <https://doi.org/10.14445/22312803/IJCTT-V70I3P104>.
- [16] Eugene Syriani, Lechanceux Luhunu y Houari Sahraoui. «Systematic mapping study of template-based code generation». En: *Computer Languages, Systems & Structures* 52 (2018), págs. 43-62. DOI: 10.1016/j.cl.2017.11.003. URL: <https://doi.org/10.1016/j.cl.2017.11.003>.
- [17] Janis Sejans y Oksana Nikiforova. «Practical Experiments with Code Generation from the UML Class Diagram». En: *Proceedings of the 3rd International Workshop on Model-Driven Architecture and Modeling-Driven Software Development (MDA & MDSD 2011)*. SciTePress, 2011, págs. 57-67. DOI: 10.5220/0003581300570067. URL: <https://doi.org/10.5220/0003581300570067>.

- [18] Lena Khaled. «A Comparison between UML Tools». En: *2009 Second International Conference on Environmental and Computer Science*. IEEE, 2009, págs. 111-114. DOI: 10.1109/ICECS.2009.38. URL: <https://doi.org/10.1109/ICECS.2009.38>.

13 Anexos

Anexo A: Código fuente completo generado

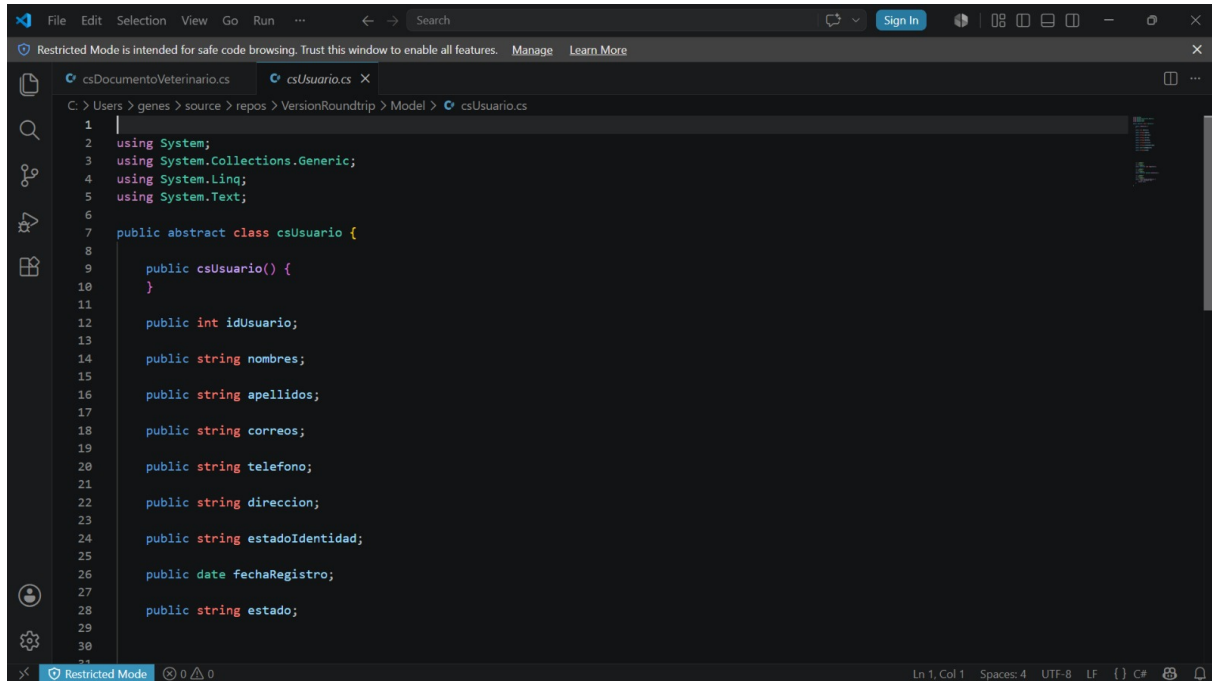


Figura 22: Código generado — csUsuario.cs

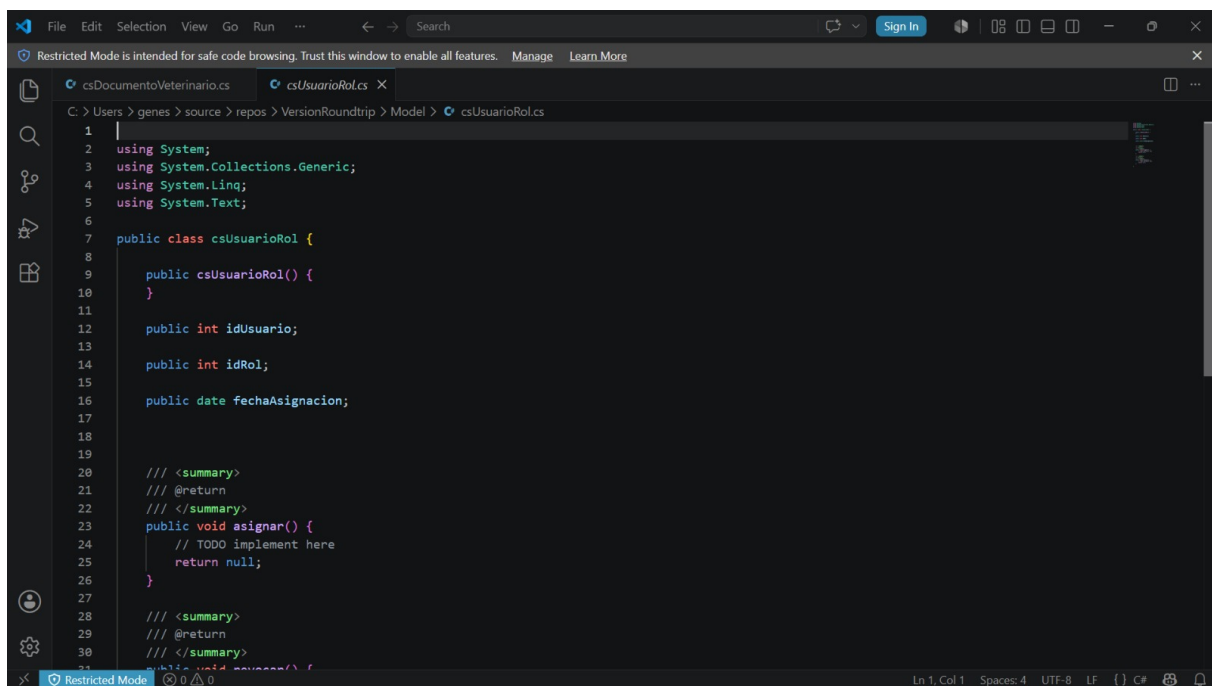


Figura 23: Código generado — csUsuarioRol.cs

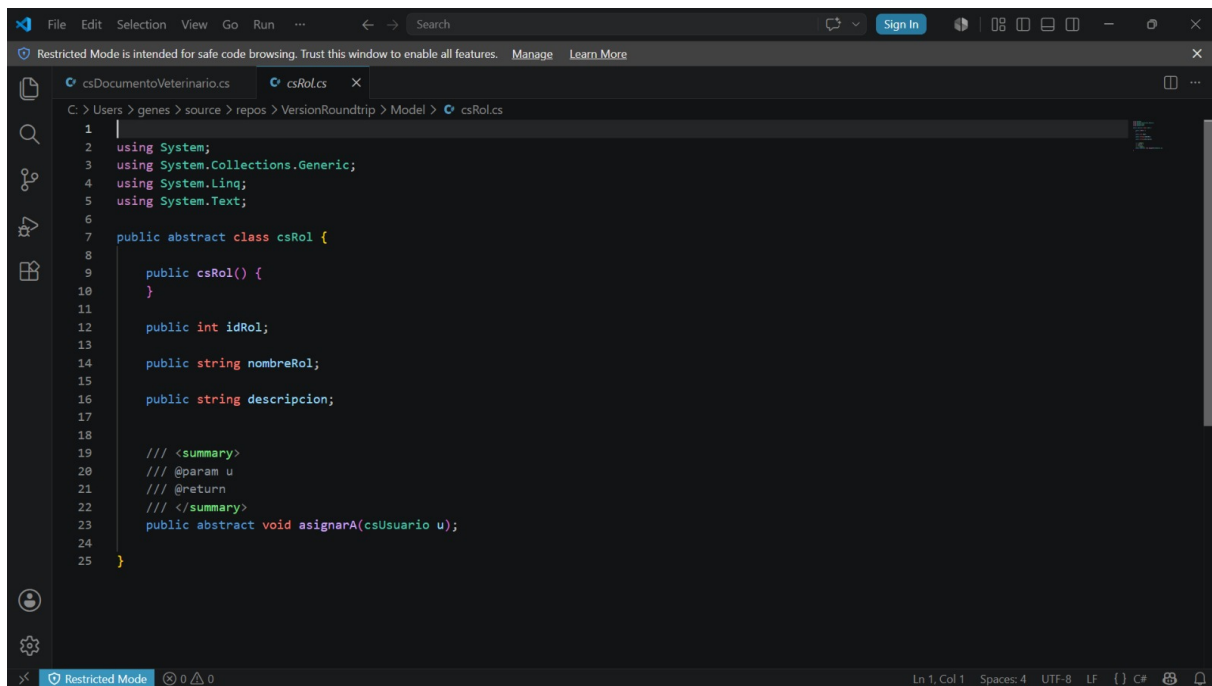


Figura 24: Código generado — csRol.cs

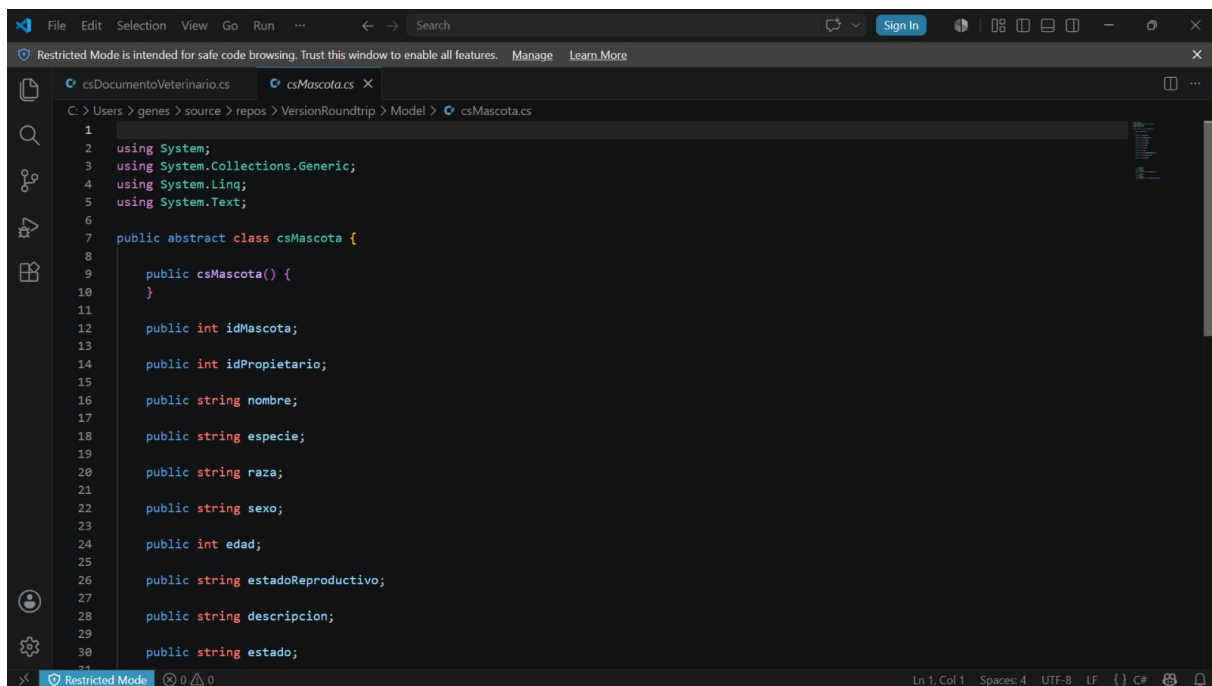
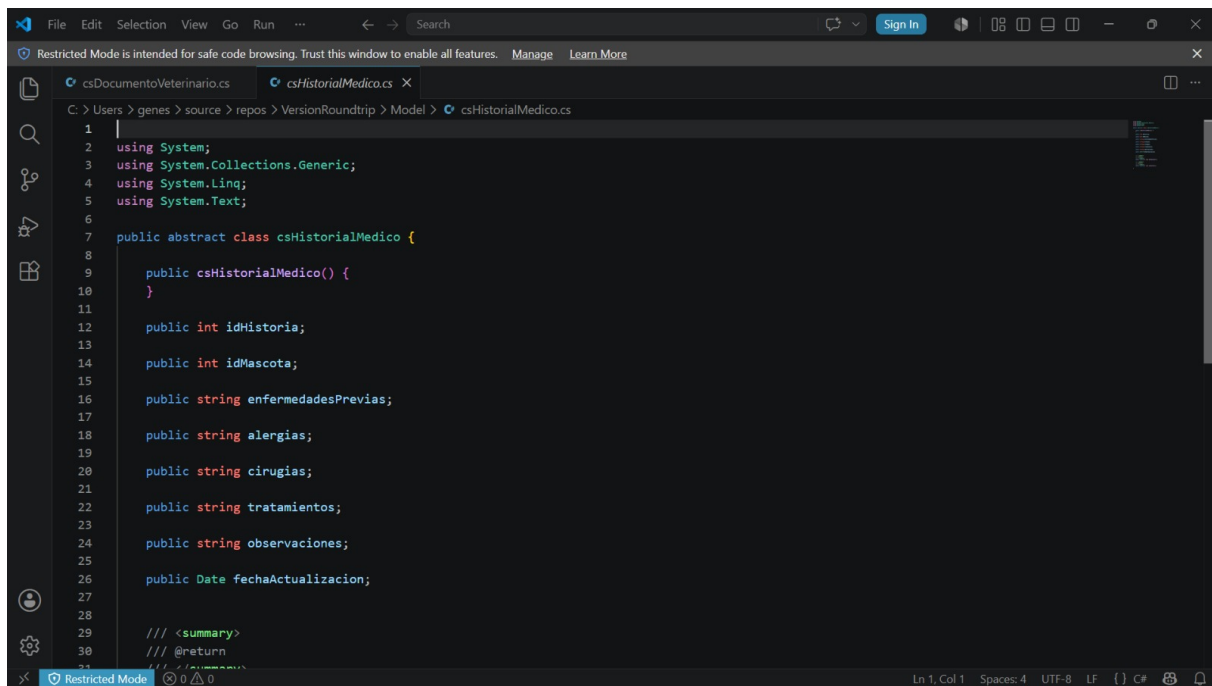
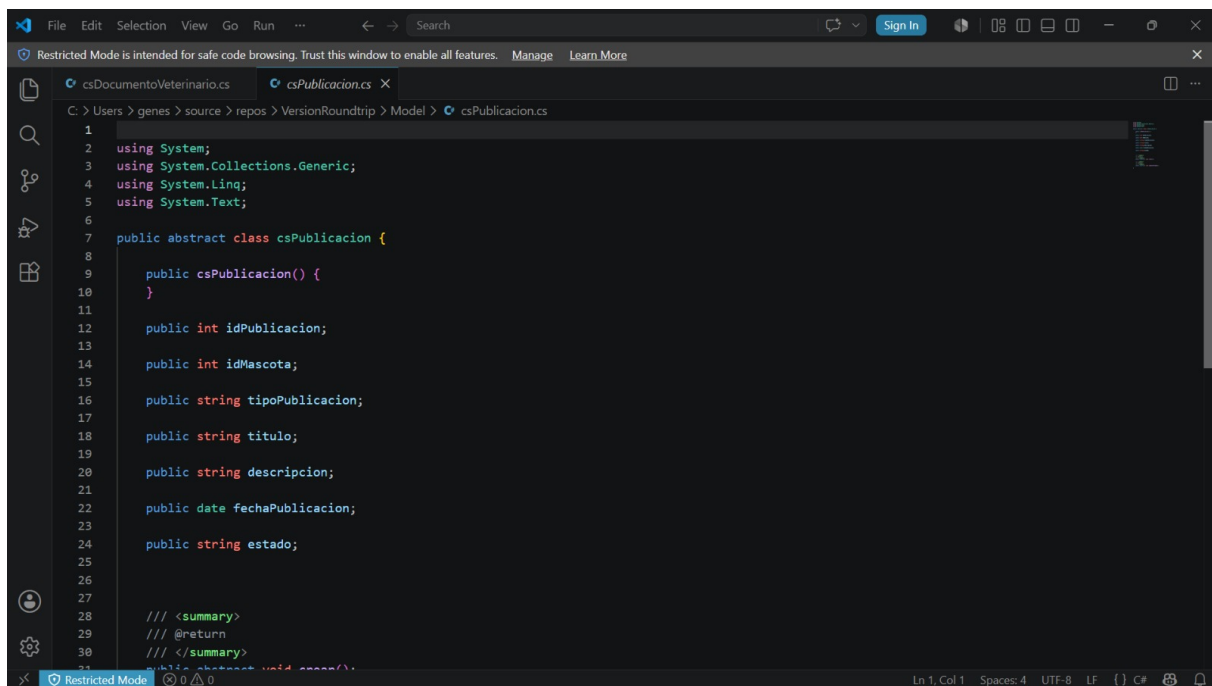


Figura 25: Código generado — csMascota.cs



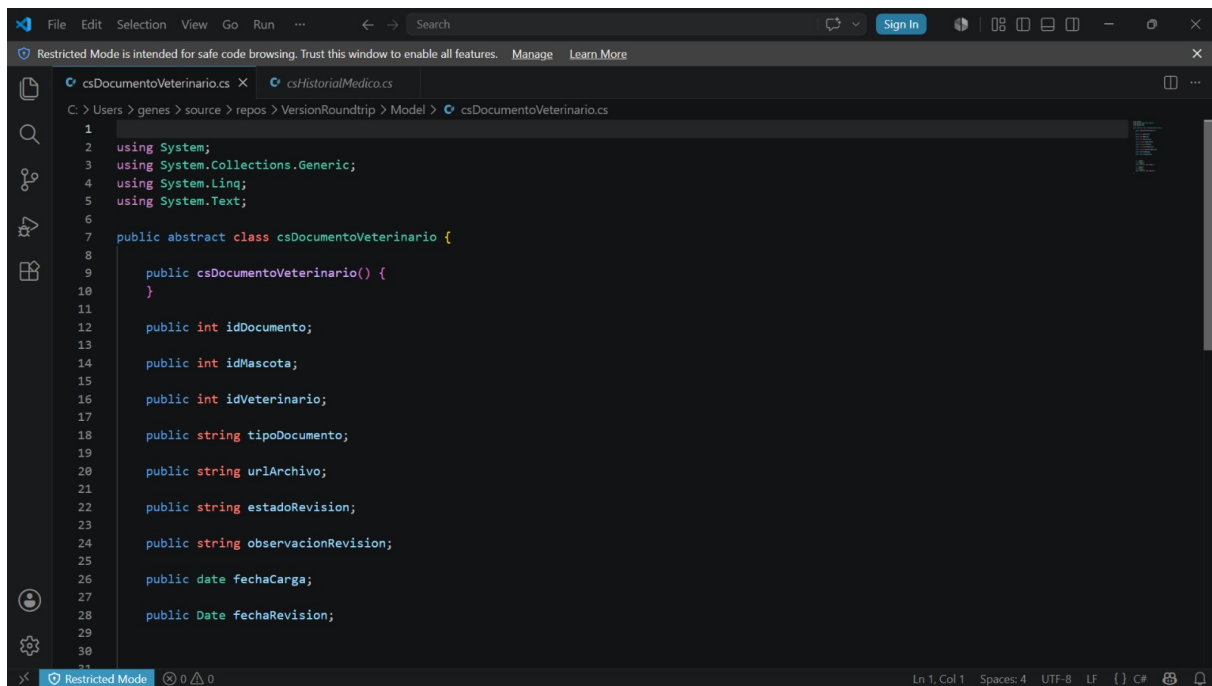
```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5
6 public abstract class csHistorialMedico {
7
8     public csHistorialMedico() {
9     }
10
11     public int idHistoria;
12
13     public int idMascota;
14
15     public string enfermedadesPrevias;
16
17     public string alergias;
18
19     public string cirugias;
20
21     public string tratamientos;
22
23     public string observaciones;
24
25     public Date fechaActualizacion;
26
27     /// <summary>
28     /// @return
29     /// </summary>
30     public abstract void crear();
31 }
```

Figura 26: Código generado — csHistorialMedico.cs



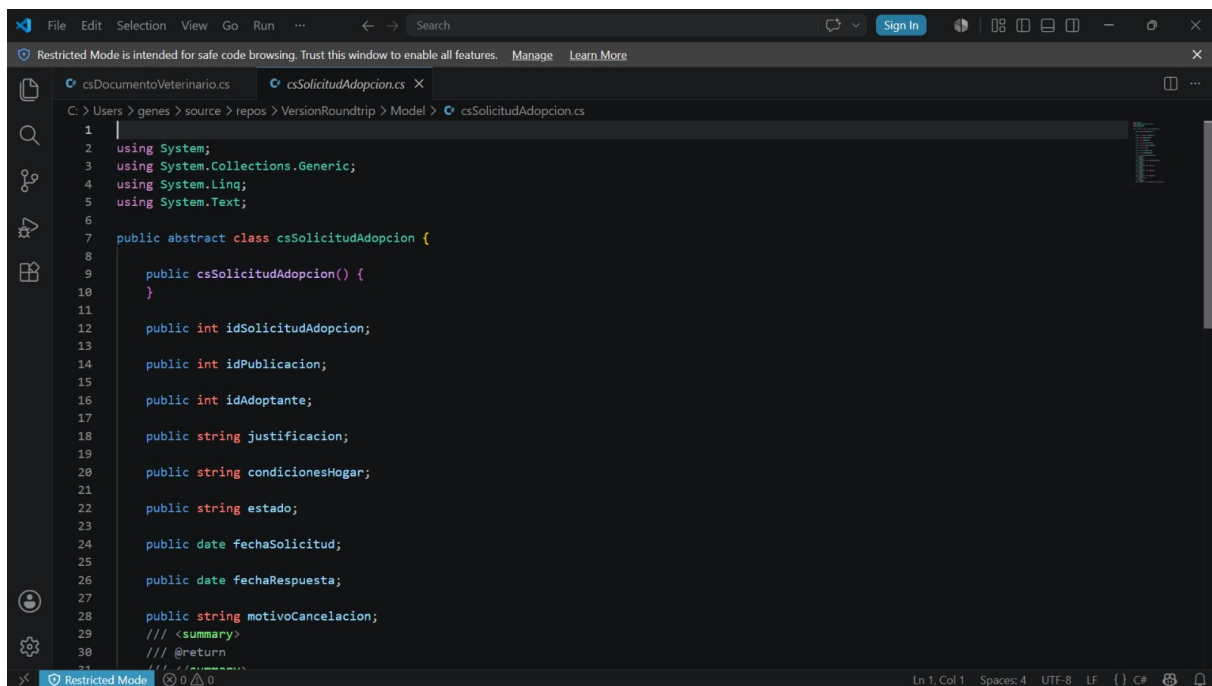
```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5
6 public abstract class csPublicacion {
7
8     public csPublicacion() {
9     }
10
11     public int idPublicacion;
12
13     public int idMascota;
14
15     public string tipoPublicacion;
16
17     public string titulo;
18
19     public string descripcion;
20
21     public date fechaPublicacion;
22
23     public string estado;
24
25     /// <summary>
26     /// @return
27     /// </summary>
28     public abstract void crear();
29 }
```

Figura 27: Código generado — csPublicacion.cs



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5
6 public abstract class csDocumentoVeterinario {
7
8     public csDocumentoVeterinario() {
9     }
10
11     public int idDocumento;
12
13     public int idMascota;
14
15     public int idVeterinario;
16
17     public string tipoDocumento;
18
19     public string urlArchivo;
20
21     public string estadoRevision;
22
23     public string observacionRevision;
24
25     public date fechaCarga;
26
27     public Date fechaRevision;
28
29
30
31 }
```

Figura 28: Código generado — csDocumentoVeterinario.cs



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5
6 public abstract class csSolicitudAdopcion {
7
8     public csSolicitudAdopcion() {
9     }
10
11     public int idSolicitudAdopcion;
12
13     public int idPublicacion;
14
15     public int idAdoptante;
16
17     public string justificacion;
18
19     public string condicionesHogar;
20
21     public string estado;
22
23     public date fechaSolicitud;
24
25     public date fechaRespuesta;
26
27     public string motivoCancelacion;
28
29     /// <summary>
30     /// @return
31 }
```

Figura 29: Código generado — csSolicitudAdopcion.cs

Anexo B: Fragmento del generador de código

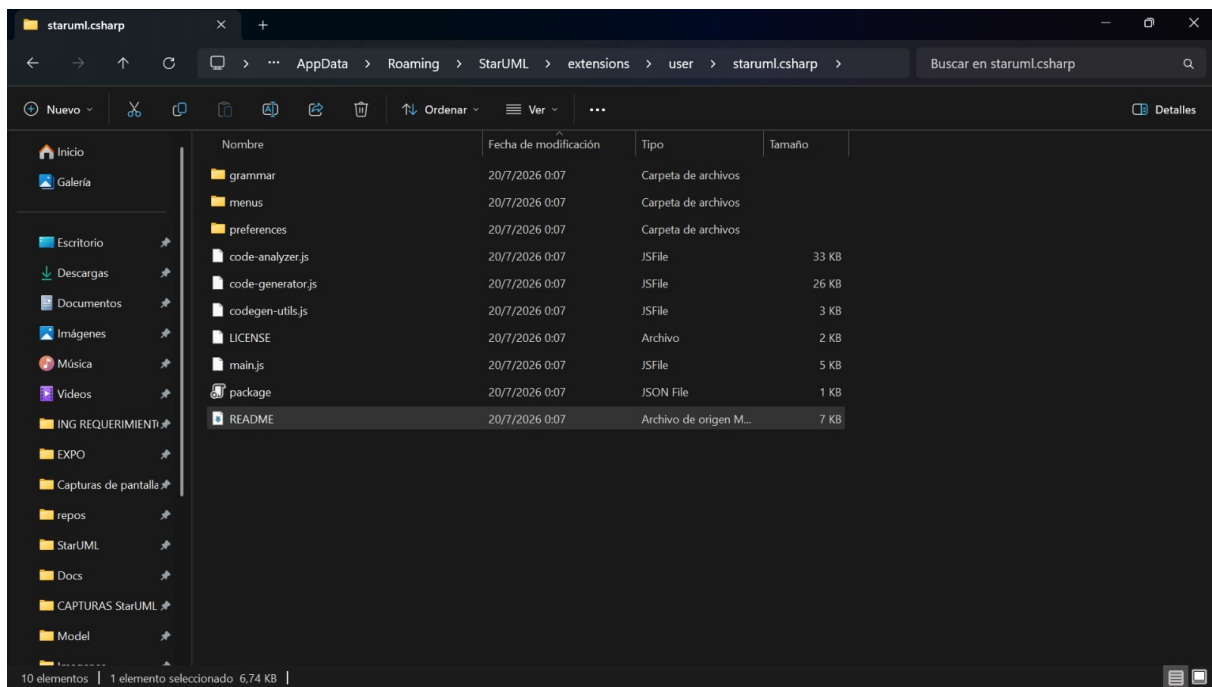


Figura 30: Fragmento de `code-generator.js` — función `getType()`